LECTURE NOTES · EFFICIENCY

Sparsity and Precision, Visually

Companion notes to the slides

The two levers that cut the cost of a language model. Mixture of experts, pruning, sparse activations and sparse attention; number formats, low-precision training, precision scaling laws, and quantization for inference. One section per slide, with every term defined and the intuition spelled out, and a reading list for each part.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *Sparsity and Precision, Visually* (33 slides) slide by slide. Numbers quoted from a paper were checked against that paper; numbers marked “derived” were computed from published model configurations; curves the slides label “illustrative” are sketches of a mechanism, not data. Claims marked † come from vendor pages, model cards or write-ups rather than papers.

Contents

How to use these notes **3**

1 Overview (slides 1–3) **3**
1.1 Slide 1: Title: Sparsity and Precision, Visually . . . 3
1.2 Slide 2: Two levers on one cost model . . . 4
1.3 Slide 3: The map, and a timeline . . . 6

2 Mixture of experts (slides 4–9) **8**
2.1 Slide 4: The mixture-of-experts layer . . . 8
2.2 Slide 5: Routing: keeping the experts busy, and fed . . . 9
2.3 Slide 6: Fine-grained experts, shared experts, and who uses what . . . 10
2.4 Slide 7: MoE systems: expert parallelism, all-to-all, and the memory bill . . . 11
2.5 Slide 8: MoE scaling laws: how sparse should a model be? . . . 13
2.6 Slide 9: A worked bill: DeepSeek-V3 against a dense model . . . 14

3 Weight pruning (slides 10–13) **16**
3.1 Slide 10: Weight pruning: the classics . . . 16
3.2 Slide 11: Structure decides the speedup . . . 17
3.3 Slide 12: One-shot pruning at LLM scale: SparseGPT and Wanda . . . 18
3.4 Slide 13: Pruning with distillation, and a law for sparsity . . . 20

4 Activation and attention sparsity (slides 14–17) **22**
4.1 Slide 14: Activation sparsity: most neurons are silent for most tokens . . . 22
4.2 Slide 15: Attention: where the cost is, and the KV cache . . . 23
4.3 Slide 16: Learned sparse attention: pick the tokens that matter . . . 24
4.4 Slide 17: The KV cache is where sparsity and precision meet . . . 26

5 Number formats (slides 18–21) **28**
5.1 Slide 18: Anatomy of a number . . . 28
5.2 Slide 19: Scale factors: where the lost bits go . . . 29
5.3 Slide 20: Outliers: the few channels that set everyone's scale . . . 31
5.4 Slide 21: What the tensor cores run, by generation . . . 32

6 Low-precision training (slides 22–24) **35**
6.1 Slide 22: Mixed-precision training: master weights, loss scaling, BF16 . . . 35
6.2 Slide 23: FP8 training: the scale goes per tensor, then per tile . . . 36
6.3 Slide 24: FP4 training arrives . . . 37

7 Scaling laws for precision (slide 25) **39**
7.1 Slide 25: Scaling laws for precision . . . 39

8 Quantization for inference (slides 26–29) **41**
8.1 Slide 26: Post-training quantization I: weights only . . . 41
8.2 Slide 27: Post-training quantization II: activations too . . . 42
8.3 Slide 28: QAT, ternary weights, and quantized experts . . . 43
8.4 Slide 29: Measuring the damage . . . 45

9 Practice (slides 30–31) 47
9.1 Slide 30: Serving three models: the bill, with every lever applied 47
9.2 Slide 31: A practical guide 47

10 Wrap-up (slides 32–33) 49
10.1 Slide 32: Summary: one cost model, two levers 49
10.2 Slide 33: Sources 49

A Notation 51

B Formulas at a glance 52

C Exercises 53

D Reading list by section 55

Index of terms 63

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the equations written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. N_{total} is the number of parameters a model stores and N_{active} the number it uses for one token; for a dense model they are equal. L is a sequence length in tokens where attention is discussed and a loss where scaling laws are discussed; the context makes it clear. Bytes are decimal (GB $= 10^9$ bytes) except for per-token cache sizes, which the slides quote in KB $= 1024$ bytes because that is how the configurations divide. FLOP/s is a rate; FLOPs is a count. A *dense* peak is a tensor-core throughput without the "with sparsity" doubling that vendors put in headlines.

A reading plan for a single lecture: Sections 1 and 2 (the cost model and mixture of experts) are the core and take about a third of the session; Section 3 (pruning) can be assigned as reading; Section 4 (activation and attention sparsity) is where the 2025–26 systems live; Sections 5–7 (formats, training, laws) are the precision half and take another third; Section 8 (quantization) is what students will actually do; Sections 9 and 10 close the loop. Appendix C is the reading list, section by section, and matches the paper archive that accompanies these notes.

1 Overview (slides 1–3)

1.1 Slide 1: Title: Sparsity and Precision, Visually

The two levers that cut the cost of a language model: mixture of experts, pruning, sparse activations and sparse attention; number formats, low-precision training, precision scaling laws, and quantization for inference.

What the slide shows A weight matrix drawn as a grid in which about half the cells are blank and the rest are filled in one of four shades. The blank cells are *pruned* weights, removed from the model; the four shades are the only values a *2-bit* number can take. The one picture contains both halves of the lecture: sparsity decides which entries exist, precision decides how finely the surviving entries are stored.

Details The deck has ten footer stages: Overview, MoE, Pruning, Activations, Formats, Training, Laws, Quantization, Practice, Wrap-up. The first four are the sparsity half, the next four the precision half, and the last two put them together on real models. The scope is the *cost* of a model, at training time and at inference time, measured in the two currencies that matter on an accelerator: floating-point operations and bytes moved. Quality enters only as the thing that must not be lost; the lecture is not about making models better, it is about making the same model cheaper, or a bigger model affordable.

The two words in the title are used in a specific sense. *Sparsity* means that, for a given token, most of the model's numbers are not touched: either because they are zero (pruning, activation sparsity), because a router decided not to use them (mixture of experts), or because a query does not look at them (sparse attention).

Precision means the number of bits used to store and multiply each number that is touched. Both are ways of doing less arithmetic and moving fewer bytes per token.

Terms

sparsity The fraction of a set of numbers (weights, activations, attention entries) that are zero or skipped. "50% sparsity" means half of them are not used. The complementary *density* is the fraction that is used.

precision The number of bits used to represent a number, and hence how finely it can be resolved. A 16-bit float distinguishes about three decimal digits; a 4-bit float distinguishes eight magnitudes.

quantization Mapping numbers from a high-precision format to a low-precision one, usually a small set of levels times a scale factor. Used both as a verb (quantizing a model) and for the result (a quantized model).

mixture of experts (MoE) A layer made of several parallel sub-networks (experts) and a router that sends each token to a few of them. The parameters of the unused experts are not touched by that token.

pruning Removing weights from a trained network, by setting them to zero or deleting whole structures, and usually retraining to recover.

Intuition

Picture a library. Sparsity is deciding which books a reader actually opens for a given question; precision is how legible the print is in the books that are opened. A reader who opens three books out of three hundred, printed clearly enough, gets the answer at a hundredth of the cost of reading everything, and the library still contains everything. The whole lecture is about how far "three" and "clearly enough" can be pushed.

1.2 Slide 2: Two levers on one cost model

Sparsity cuts how many numbers each token touches; precision cuts how many bits each number has.

What the slide shows A roofline plot for one H100 SXM GPU. The horizontal axis is *arithmetic intensity*, FLOPs performed per byte read from high-bandwidth memory; the vertical axis is the throughput the chip can deliver. The roof has two parts: a sloped line, throughput $=$ 3.35 TB/s $\times$ intensity, where the chip is waiting on memory; and a flat line at 989 TFLOP/s, its dense BF16 peak, where it is doing arithmetic as fast as it can. The two meet at the *ridge*, $989/3.35 \approx 295$ FLOP per byte. Four points are placed on it: decoding one token at batch 1 in BF16 sits at 1 FLOP/byte; the same with 4-bit weights at 4; a batch of 64 tokens at 64; a 4k-token prefill far to the right. The right panel is the accounting: four formulas and three bullets.

Details The formulas are the cost model used throughout.

$$\text{training FLOPs} \approx 6 N_{\text{active}} D \qquad \text{decode FLOPs per token} \approx 2 N_{\text{active}}$$

$$\text{weight bytes} = N_{\text{total}} \cdot \text{bits}/8 \qquad \text{KV-cache bytes per token} = 2 n_{\text{layers}} h_{kv} d_h \cdot \text{bits}/8$$

The factor 6 is the usual count for a dense transformer: 2 FLOPs per parameter per token in the forward pass (one multiply, one add) and 4 in the backward pass. The factor 2 for decoding is the forward pass alone. The point of writing N_{active} rather than N is that for a mixture of experts the two differ, by a factor of eighteen in DeepSeek-V3.

The roofline argument is the heart of the lecture. When a model generates one token, every weight it uses must be read from memory once, and each weight participates in one multiply-add, i.e. 2 FLOPs. At 16 bits a

weight is 2 bytes, so the intensity is $2/2 = 1$ FLOP per byte. The chip can multiply 295 times faster than that memory traffic allows: at batch 1, decoding uses about a third of a percent of the arithmetic capacity of an H100. The step time is simply (bytes of active weights)/(bandwidth): for a 70B model in BF16, 140 GB at 3.35 TB/s is about 42 ms per token, however little arithmetic is involved.

This is why the two levers work. Four-bit weights divide the bytes by four and move the point to 4 FLOP/byte, still memory-bound, so the step is four times faster. A mixture of experts that touches 37B of its 671B parameters reads 37 GB at FP8 instead of 671 GB. Batching B tokens shares each weight read among B tokens and moves the point right by B; at $B \approx 300$ the layer reaches the ridge and becomes compute-bound, after which weight precision stops helping and only arithmetic throughput (and the precision of the arithmetic) matters. Prefill, which processes thousands of prompt tokens at once, lives on the flat part of the roof from the start.

The catch in the third bullet is the lecture's running theme. Rounding and removal cost quality, in ways that are often invisible to perplexity and visible on hard tasks (slide 29). And a number that is zero, or small, saves nothing unless the hardware or the kernel can skip it (slide 11) or multiply it natively (slide 21).

Terms

roofline model A plot of attainable throughput against arithmetic intensity, bounded by the memory-bandwidth line and the peak-compute line (Williams, Waterman and Patterson, 2009). A workload's position tells whether it is memory-bound or compute-bound.

arithmetic intensity FLOPs performed per byte moved from memory, for a given operation. High intensity means the operation can keep the arithmetic units busy; low intensity means it waits on memory.

ridge point The intensity at which the memory-bound and compute-bound limits meet, equal to peak FLOP/s divided by bandwidth. Below it the workload is memory-bound.

memory-bound Limited by the rate at which data can be moved, not by the rate at which it can be computed on. Decoding at small batch is the canonical case.

compute-bound Limited by arithmetic throughput. Prefill and large-batch decoding are compute-bound.

HBM High-bandwidth memory, the stacked DRAM on the GPU package. Its bandwidth (3.35 TB/s on an H100 SXM, 8 TB/s on a B200) is the denominator of every decode-time estimate.

decode The phase of generation in which tokens are produced one at a time, each requiring a full forward pass over the active weights and a read of the KV cache.

prefill The phase in which the prompt is processed, all tokens in parallel; it fills the KV cache and is compute-bound.

batch The number of independent sequences (or tokens) processed together in one forward pass. Batching shares each weight read across the batch and raises arithmetic intensity.

active parameters N_{active}: the parameters used to process one token. Equal to the total for a dense model; the chosen experts plus everything else for a mixture of experts.

KV cache The keys and values of every previous token in every attention layer, stored so that they need not be recomputed at each decoding step. It grows linearly with the sequence length.

Intuition

At batch 1 a GPU decoding a language model is a very fast calculator attached to a very slow bookshelf. Nothing you do to the calculator helps; everything you do to the number of books, or to their thickness, helps in direct proportion. Sparsity is fewer books per question; precision is thinner books. When the question changes to "serve a thousand users at once", the bookshelf is shared and the calculator matters again, and the two levers change roles.

1.3 Slide 3: The map, and a timeline

Four kinds of sparsity, four questions about precision, and when each arrived in language models.

What the slide shows Two rows of four cards and a timeline. The sparsity row: mixture of experts (trained in, a router picks k of E experts per token); weight pruning (after training; zero out weights, structured or not); activation sparsity (per input, most MLP neurons are silent); attention sparsity (per query, read a subset of the KV cache). The precision row: formats; training in low precision; scaling laws for precision; quantization for inference. The timeline runs from magnitude pruning and mixed-precision training (2015–17) through GShard and Switch (2020), LLM.int8 and GPTQ and the FP8 formats (2022), SparseGPT, AWQ, the MX specification and Mixtral (2023), DeepSeek-V3's FP8 training and the precision scaling law (2024), native sparse attention, Blackwell's FP4 and NVFP4 pretraining (2025), to NVFP4 pretraining of a 550B model and compressed sparse attention in 2026 (marked †, from reports and model cards).

Details The four sparsities differ in *when* the decision not to touch a number is made and *who* makes it.

- Mixture of experts: the decision is made by a learned router at every token and every MoE layer, and the model is trained from the start with that structure. The skipped parameters are large contiguous blocks (whole experts), which is what makes the skipping efficient.
- Weight pruning: the decision is made once, after training, per weight or per structure; the zeros are permanent and input-independent.
- Activation sparsity: the decision is made per input by the activation function (a ReLU that outputs zero) or a threshold; the zeros are input-dependent but not learned as a separate mechanism.
- Attention sparsity: the decision is made per query over the past tokens, by a fixed pattern, by an eviction rule, or by a learned selector.

The precision questions are ordered by dependency. Formats (what the bits mean) come first; training in low precision needs the formats plus scale factors and rounding rules; the scaling laws say how much precision is worth against parameters and data; quantization for inference applies all of it to a finished model.

The timeline makes one historical point: the sparsity ideas are old (pruning is from the 1980s, mixtures of experts from 1991, magnitude pruning of deep networks from 2015) and the precision ideas arrived with the hardware that could use them (FP16 tensor cores in 2017, BF16 in 2020, FP8 in 2022, FP4 in 2025). What changed in 2023–26 is that the two met at the frontier: the models that set records are now sparse and low-precision by construction.

Terms

router A small learned function (usually one linear layer and a softmax or sigmoid) that assigns each token to experts in an MoE layer.

expert One of the parallel sub-networks in an MoE layer; in transformers, a feed-forward (MLP) block.

structured sparsity Sparsity whose zeros follow a pattern the hardware can exploit: whole rows, columns, blocks, or a fixed ratio within small groups (such as 2:4).

unstructured sparsity Zeros at arbitrary positions. Highest quality per removed weight, hardest to speed up.

MX, NVFP4 Block floating-point formats in which a group of 32 (MX) or 16 (NVFP4) low-bit numbers share one scale factor. Defined on slide 19.

Intuition

Keep the two rows separate in your head. The sparsity row answers "which numbers?" and is mostly about architecture and algorithms. The precision row answers "how many bits?" and is mostly about hardware and numerics. They multiply: a model can be both, and the frontier models of 2025 are.

2 Mixture of experts (slides 4–9)

2.1 Slide 4: The mixture-of-experts layer

Replace one wide MLP by E narrow ones and a router; each token uses k of them, so FLOPs follow k, not E.

What the slide shows A token x enters a router, which produces a score for each of eight experts; the bar chart of scores shows two tall bars (experts 2 and 5) and six short ones. Only the two chosen experts run; their outputs are weighted by the scores and summed, and the sum is added to the residual stream. The formula below gives the layer; the panel at the right counts parameters: total versus active.

Details An MoE layer replaces the feed-forward block of a transformer layer. Write the router weights as $W_r \in \mathbb{R}^{E\times d}$ and the experts as functions $E_i : \mathbb{R}^d \to \mathbb{R}^d$, each a small MLP.

$$g = \mathrm{softmax}(W_r x) \in \mathbb{R}^E, \qquad \mathcal{S} = \mathrm{TopK}(g), \qquad y = x + \sum_{i\in\mathcal{S}} g_i E_i(x).$$

The gates g_i for the chosen experts are often renormalized to sum to one; DeepSeek-V3 and Kimi K2 use a sigmoid instead of a softmax and normalize over the chosen set. Only the k selected experts are evaluated, so the FLOPs of the layer are those of k small MLPs plus the router, independent of E. The parameters of all E experts are stored, so the memory is that of E MLPs.

Parameter counting. N_{total} counts everything: attention, embeddings, and all experts. N_{active} counts attention, embeddings, the router, and k experts (plus any shared expert). DeepSeek-V3: $N_{\text{total}} = 671\text{B}$, $N_{\text{active}} = 37\text{B}$, a ratio of about 18. For a dense model the two are equal.

Why it works at all is an empirical question that the scaling-law papers (slide 8) try to answer. The short version: the quality of a language model is set mainly by how much it can store and by how many parameters participate in each token, and the first of these matters more than the second. An MoE with N_{total} parameters is better than a dense model with N_{active} parameters and worse than a dense model with N_{total}; the gap to the latter is the "discount" in the slide's last line, and it has been shrinking as routing and expert granularity improved.

The idea is old. Jacobs, Jordan, Nowlan and Hinton (1991) introduced adaptive mixtures of local experts; Shazeer et al. (2017) made it sparse and large ("outrageously large") inside an LSTM language model with a noisy top-k gate; GShard (2020) and Switch Transformer (2021) put it in transformers at scale.

Terms

top-k routing Selecting the k experts with the largest router scores for a token; $k = 1$ (Switch), 2 (GShard, Mixtral), 4 (gpt-oss), 8 (DeepSeek-V3, Qwen3, Kimi K2).

gate The weight g_i with which an expert's output is added; also the router's output as a whole.

feed-forward block (MLP) The position-wise two-layer network in a transformer layer: $W_{\text{down}}\,\sigma(W_{\text{up}}x)$, often with a gated variant (SwiGLU). It holds about two thirds of a dense transformer's parameters and is what MoE replaces.

residual stream The vector carried from layer to layer to which each sub-layer adds its output. MoE outputs are added to it like any other sub-layer.

total parameters N_{total}: everything stored, including all experts.

sigmoid gate A router whose scores are $\sigma(W_r x)$ per expert, independently, rather than a softmax across experts; used in DeepSeek-V3 to decouple the selection from the normalization.

Intuition

A dense MLP is a committee in which everyone votes on everything. An MoE is a committee in which a receptionist sends each question to the two members who know the subject. The building needs offices for everyone (memory), but each question costs two people's time (FLOPs). The receptionist is the hard part: left alone, she sends every question to the same two people, which is slide 5.

2.2 Slide 5: Routing: keeping the experts busy, and fed

Left alone, a router sends most tokens to a few experts. Every recipe adds a pressure toward balance.

What the slide shows Two histograms of tokens per expert in a batch: *collapsed*, with one expert receiving most tokens and the rest almost none; *balanced*, with all eight receiving similar counts. Below, three balancing mechanisms with formulas. The right panel covers capacity factors and token dropping, expert-choice routing, and the top-k choices of the open models.

Details *Why collapse happens.* A router is trained by the same gradient as the rest of the model. Early in training, whichever expert happens to be slightly better receives more tokens, trains faster, becomes better still, and receives more. Without a counter-pressure the system settles into a state where a few experts do all the work and the others are dead weight: their parameters are never updated and never used. This wastes most of N_{total} and overloads the few live experts.

Auxiliary load-balancing loss (Switch Transformer, following GShard). For a batch of T tokens, let f_i be the fraction of tokens routed to expert i and P_i the mean router probability assigned to expert i over the batch.

$$\mathscr{L}_{\text{aux}} = \alpha E \sum_{i=1}^{E} f_i P_i, \quad \text{added to the language loss, with } \alpha = 0.01 \text{ in Switch.}$$

The sum $\sum_i f_i P_i$ is minimized when both vectors are uniform ($f_i = P_i = 1/E$), where it equals $1/E$; the factor E makes the minimum 1 regardless of E. The product form is used because f_i is not differentiable (it is a count) while P_i is, so the gradient flows through P_i weighted by the observed load.

Router z-loss (ST-MoE). $\mathscr{L}_z = \frac{1}{T}\sum_t \left(\log \sum_j e^{x_{tj}}\right)^2$ penalizes large router logits, which otherwise drift upward and make the softmax saturate and the routing unstable, especially in low precision.

Auxiliary-loss-free balancing (Wang et al. 2024; DeepSeek-V3). Each expert carries a bias b_i that is added to its score *only for the purpose of top-k selection*; the gate value used in the sum is the unbiased score. After each training step, the load of every expert is measured; overloaded experts have b_i decreased by γ, underloaded ones have it increased by γ. The bias steers tokens toward idle experts without adding a term to the loss, so the language-modeling gradient is undisturbed. DeepSeek-V3 reports better quality than the auxiliary-loss approach at the same balance.

Expert-choice routing (Zhou et al. 2022) inverts the assignment: each expert selects the top tokens for itself from the batch, so every expert receives exactly the same number. Balance is perfect by construction and tokens can be used by a variable number of experts. The drawback is that a token's routing depends on which other tokens are in the batch, which is natural in training but awkward at decoding, where a token arrives alone.

Capacity and dropping. Static kernels want a fixed number of tokens per expert per batch. The *capacity* is $C = \text{CF} \cdot T/E$ with a *capacity factor* CF slightly above 1 (1.0–1.25 in Switch and GShard). Tokens assigned to an expert beyond its capacity are *dropped*: they skip the expert and pass through on the residual connection. Dropping wastes the token's compute in that layer and injects noise; *dropless* MoE (MegaBlocks, 2022) avoids it by expressing the expert computation as a block-sparse matrix multiplication whose blocks can have variable sizes, so each expert processes exactly the tokens routed to it.

Top-k in practice. $k = 1$ was Switch's choice for simplicity; $k = 2$ was GShard's and Mixtral's; the 2024–25 generation uses $k = 8$ with many small experts (next slide), and gpt-oss uses 4 of 128.

Terms

router collapse The failure mode in which most tokens are routed to a few experts, leaving the rest untrained and unused.

auxiliary loss A term added to the training objective to encourage a property (here, balanced load) that the main loss does not reward.

load balancing Any mechanism that keeps the number of tokens per expert roughly equal, so that all experts train and no expert overflows.

z-loss A penalty on the log-partition function of the router's softmax, keeping logits small and routing stable.

capacity factor (CF) The multiple of the average load that an expert's buffer can hold. CF $= 1.25$ means each expert can take 25% more than its share before dropping.

token dropping Skipping an expert for tokens that exceed its capacity; the token continues through the residual connection unchanged by that layer.

dropless MoE An implementation (MegaBlocks) that handles variable numbers of tokens per expert with block-sparse kernels, so no token is dropped.

expert choice Routing in which experts pick tokens rather than tokens picking experts; exact balance, batch-dependent assignment.

Intuition

Balance is a stability problem, not a quality problem: an unbalanced router is a positive feedback loop, and the mechanisms on this slide are all ways to add negative feedback. The auxiliary loss adds it through the gradient and perturbs the language objective; the bias method adds it outside the gradient; expert choice removes the loop by construction. The trend from 2021 to 2025 is toward the methods that interfere least with the language loss.

2.3 Slide 6: Fine-grained experts, shared experts, and who uses what

Same FLOPs, far more combinations; and the active fraction fell from a quarter to a thirtieth in two years.

What the slide shows Two expert arrays at the same active FLOPs: eight coarse experts with two active, giving $\binom{8}{2} = 28$ possible combinations; thirty-two experts of a quarter the width with eight active, giving $\binom{32}{8} = 10{,}518{,}300$ combinations; plus a green "shared" expert that is always on. Below, a table of open MoE models from Mixtral (December 2023) to Qwen3-Next (September 2025) with total and active parameters, their ratio, expert counts and the number active per token.

Details *Granularity.* DeepSeekMoE (Dai et al. 2024) made two changes to the GShard design. First, split each expert into m smaller ones and activate mk of them, keeping the active width (hence FLOPs) fixed while multiplying the number of distinct expert combinations a token can use. The combinatorial count on the slide is the simplest way to see the gain: the model can express far more specialized "teams" of experts for the same compute. Krajewski et al. (2024) formalized this as *granularity* G, the ratio of the dense MLP width to an expert's width, and found that the standard choice ($G = 1$, experts as wide as the dense MLP they replace) is not optimal at almost any compute budget; finer is better, up to the point where the router and the all-to-all communication (slide 7) dominate.

Shared experts. Second, DeepSeekMoE isolates K_s experts that every token uses, in addition to its routed ones. The idea is that some computation is needed by all tokens (common syntax, formatting) and would

otherwise have to be duplicated in many routed experts. DeepSeek-V3 uses 1 shared plus 8 of 256 routed; Llama 4 uses 1 shared plus 1 routed of 128; Kimi K2, 1 shared plus 8 of 384; Qwen3-Next, 1 shared plus 10 of 512. Qwen3's flagship MoE deliberately has no shared expert, so the choice is not universal.

The table, with the sources the slide relies on (model cards and technical reports):

model (date)	total	active	active/total	experts per layer	per token
Mixtral 8×7B (Dec 2023)	47B	13B	28%	8	2
DeepSeek-V3 (Dec 2024)	671B	37B	5.5%	256 + 1 shared	8 + 1
Llama 4 Maverick (Apr 2025)	400B	17B	4.3%	128 + 1 shared	1 + 1
Qwen3-235B-A22B (Apr 2025)	235B	22B	9.4%	128	8
Kimi K2 (Jul 2025)	1.04T	32B	3.1%	384 + 1 shared	8 + 1
gpt-oss-120b (Aug 2025)	117B	5.1B	4.4%	128	4
Qwen3-Next-80B-A3B (Sep 2025)	80B	3B	3.8%	512 + 1 shared	10 + 1

The ratio column is derived. Two things to notice. The active fraction fell by an order of magnitude in two years, from Mixtral's 28% to the 3–5% of the 2025 models, and the way it fell was by adding experts rather than by shrinking the active set: k went up (2 to 8 or 10) while expert width went down. Llama 4 is the exception that proves the rule, with coarse experts and $k = 1$, and it alternates dense and MoE layers. The 2026 releases continue the trend (†): DeepSeek-V4-Pro at 1.6T total and 49B active, Kimi K3 at 2.8T total and 104B active with 16 of 896 experts plus two shared.

Terms

fine-grained experts Experts narrower than the dense MLP they replace, used in larger numbers with larger k so that the active FLOPs are unchanged but the number of possible expert combinations is far larger.

granularity (G) Expert width relative to the dense MLP width (Krajewski et al. 2024); $G = 4$ means experts a quarter as wide. A hyperparameter of MoE scaling laws.

shared expert An expert that every token uses regardless of routing, carrying computation common to all tokens so that it need not be duplicated across routed experts.

active fraction $N_{\text{active}}/N_{\text{total}}$, the share of parameters a token uses. The slide's ratio column.

MoE layer One transformer layer's feed-forward block replaced by experts; "256 experts" means 256 per such layer, and most layers are MoE layers in the 2025 models.

Intuition

The combinatorial argument is the one to remember. With coarse experts, specialization is coarse: an expert must be good at a large bundle of things. With fine experts the router composes a bespoke bundle per token out of many small pieces, which is both more expressive and easier to balance. The price is a longer list of experts to shuffle tokens among, which is the next slide.

2.4 Slide 7: MoE systems: expert parallelism, all-to-all, and the memory bill

The experts live on different GPUs; every MoE layer is two network shuffles around a small matmul.

What the slide shows Four GPUs, each holding its share of tokens (top row) and two of the eight experts (bottom row). Every token must travel to the GPUs holding its chosen experts (dispatch, an all-to-all collective), be processed there, and come back (combine, a second all-to-all). The two labels at the right make the central point: compute follows N_{active}, memory follows N_{total}. Three cards below cover memory, decode batching, and load at inference.

Details *Expert parallelism.* Dense models are split across GPUs by tensor parallelism (each GPU holds a slice of every matrix) or pipeline parallelism (each GPU holds some layers). MoE adds a third axis: each GPU holds some of the experts, whole. Since a token's experts are not known until the router runs, the tokens must be exchanged after routing. The all-to-all collective is the natural primitive: every GPU sends to every other GPU the tokens destined for that GPU's experts, and the reverse after the experts run. Two all-to-alls per MoE layer per step, each moving the full activation tensor across the interconnect, is the main cost that MoE adds to a training step.

DeepSeek-V3's handling of this is a good example of the engineering. Routing is *node-limited*: each token is sent to experts on at most $M = 4$ nodes, chosen among the nodes that hold its highest-scoring experts, so that the cross-node traffic is bounded. The all-to-alls are overlapped with computation by a pipeline schedule (DualPipe) that interleaves the forward and backward passes of different micro-batches so that one is communicating while another computes. The report describes this as keeping the all-to-all "almost fully overlapped".

Memory. Every expert must be resident on some GPU, so the memory needed to serve an MoE is set by N_{total}: 671B parameters at one byte each (FP8) is 671 GB, which is nine or more 80 GB GPUs before counting the KV cache. Only 37B of those parameters are used per token. MoE therefore trades memory for compute; it is the right trade when memory is available and compute per token is the constraint, which is the situation of a large serving deployment, and the wrong one on a single consumer GPU, where a dense model of the same memory footprint would be a better model.

Batching. At batch 1 a token reads only its k experts, so the bytes per token are a small fraction of N_{total} and decode is cheap. As the batch grows, different tokens choose different experts, and once the batch is large enough that every expert is used by someone, the whole N_{total} is read per step: the layer behaves like a dense layer of size N_{total} in bytes, while still computing only N_{active} per token. The transition happens at a batch of roughly E/k tokens, i.e. a few dozen for the 2025 configurations.

Load at inference. Expert popularity is uneven and domain-dependent (code prompts hit different experts from chat prompts). Serving systems replicate the hot experts, place experts so that the traffic to each GPU is similar, and re-balance periodically; DeepSeek's serving design uses *redundant experts* for this.

Terms

expert parallelism (EP) Placing different experts of the same layer on different GPUs, with all-to-all communication to route tokens to them.

all-to-all A collective communication in which every participant sends a distinct chunk of data to every other participant. The dispatch and combine steps of an MoE layer.

tensor parallelism Splitting each weight matrix across GPUs so that all GPUs cooperate on every matmul; needs an all-reduce per layer.

pipeline parallelism Placing different layers on different GPUs and passing activations along; needs micro-batching to keep all GPUs busy.

node-limited routing Restricting the experts a token may use to those on at most M nodes, to bound cross-node traffic (DeepSeek-V3, $M = 4$).

DualPipe DeepSeek-V3's pipeline schedule that overlaps the all-to-all communication of one micro-batch with the computation of another.

redundant experts Extra copies of frequently used experts placed on additional GPUs to even out the load at serving time.

Intuition

An MoE at scale is a distributed system before it is a neural network. The matmuls are small; the shuffles are what the engineers spend their time on. If you remember one number from this slide, make it the pair "37B computed, 671 GB resident": the gap between them is both the benefit and the bill.

2.5 Slide 8: MoE scaling laws: how sparse should a model be?

At a fixed training budget there is a best active fraction, and it falls as the total parameter count grows.

What the slide shows Loss against active fraction (dense at the right, sparser to the left, log scale) at a fixed training-FLOP budget, for three total parameter counts N, $4N$, $16N$. Each curve has a minimum, and the minimum moves left as the model grows. The curves are illustrative, drawn after Abnar et al. (2025). The right panel summarizes four papers and the practical consensus.

Details The question a scaling law for MoE must answer has one more dimension than the dense case: given a budget, how should it be split between total parameters (capacity), active parameters (compute per token), and tokens (data)? Four papers mark the progress.

Clark et al. 2022 ("Unified scaling laws for routed language models"), from DeepMind, fit a law in the dense parameter count and the number of experts E for three routing methods at up to 130B tokens. The returns from more experts diminish; the paper recommends $E \in \{64, 128\}$ at the scales studied, with fitted saturation points of a few hundred experts, and finds that the advantage of routing shrinks as the base model grows (at the training length used, the fits predict it vanishing somewhere near a trillion dense parameters for one router, far lower for the others). These numbers reflect the coarse-expert, short-training regime of 2022 and should not be read as limits on today's designs.

Krajewski et al. 2024 ("Scaling laws for fine-grained mixture of experts") add granularity as a variable. Their finding is that with granularity tuned, MoE outperforms dense transformers at every scale they studied, and that the gap widens with scale; the conventional expert width ($G = 1$) is suboptimal at almost any budget.

Abnar et al. 2025 ("Parameters vs FLOPs") treat total parameters and FLOPs per token as independent axes and define sparsity as $1 - N_{\text{active}}/N_{\text{total}}$. Their clean result is the one drawn on the slide: for a fixed compute budget, the optimal sparsity increases with the total model size and tends toward 1. Whether the optimum also moves with the training budget at fixed total size is stated in two opposite ways in different parts of the paper, and the slide says so rather than picking a side.

Ludziejewski et al. 2025 ("Joint MoE scaling laws") add memory to the objective and conclude that, contrary to the conventional view, an MoE can be the memory-efficient choice as well as the FLOP-efficient one, once the number of active parameters and the training length are chosen jointly.

The practical consensus in 2025, visible in the table of slide 6: $k = 8$ active out of 128–384 fine-grained experts, one shared expert, an active fraction of 3–10%. Mixtral's 2-of-8 is now the coarse end of the range.

Terms

MoE scaling law A fitted relationship between loss and the resources of an MoE model (total parameters, active parameters, experts, granularity, tokens), used to choose the architecture for a budget.

compute-optimal sparsity The active fraction that minimizes loss for a given training budget and total size.

routing benefit The loss improvement of a routed model over a dense model of the same active size; it shrinks as the dense model grows, at fixed training length.

Intuition

The curves on the slide are a bargain between two ways to spend a FLOP budget: on more parameters that are rarely used, or on using the parameters you have more thoroughly. Big models have more to gain from rarely-used parameters, because the long tail of knowledge they can hold is long. That is why the optimum drifts toward sparser as models grow, and why the frontier models of 2025 are twenty to thirty times larger than what they compute per token.

2.6 Slide 9: A worked bill: DeepSeek-V3 against a dense model

Sparsity in the experts, compression in the attention, FP8 everywhere: the three levers in one model.

What the slide shows Four tiles: 671B total parameters (256 routed + 1 shared experts per layer), 37B active per token, 14.8T training tokens in 2.788M H800 GPU-hours in FP8, and 70 KB of KV cache per token (MLA, BF16, derived). A bar chart of bytes read per decoded token at batch 1: Llama 3.1 405B in BF16, 810 GB; the same in 4-bit, 203 GB; DeepSeek-V3 in FP8 reading only its active experts, 37 GB. A note on the KV cache (504 KB per token for the 405B model against 70 KB) and a panel reading the bill.

Details All the derived numbers follow from the published configurations.

- Training FLOPs: $6N_{\text{active}}D = 6 \times 37 \times 10^9 \times 14.8 \times 10^{12} \approx 3.3 \times 10^{24}$. A dense 405B model trained on the same tokens would cost $6 \times 405 \times 10^9 \times 14.8 \times 10^{12} \approx 3.6 \times 10^{25}$, eleven times more. (The $6ND$ count ignores attention FLOPs and the multi-token-prediction head, so both numbers are slightly low in the same way.)
- Weight bytes: 671 GB at FP8; 1.34 TB at BF16. The model trained in FP8 (slide 23), so FP8 is its native serving precision.
- Bytes per token at batch 1: a dense model reads all its weights, $405 \times 10^9 \times 2$ bytes $= 810$ GB in BF16, 203 GB at 4 bits. DeepSeek-V3 reads its active parameters only: $37 \times 10^9 \times 1$ byte $= 37$ GB. At 3.35 TB/s these are 240 ms, 60 ms and 11 ms per token respectively, before the KV cache and before any batching.
- KV cache per token. Llama 3.1 405B: 126 layers, 8 KV heads, $d_h = 128$, keys and values, 2 bytes: $2 \times 126 \times 8 \times 128 \times 2 = 516{,}096$ bytes $= 504$ KB. DeepSeek-V3 with multi-head latent attention: per layer one 512-dimensional latent plus a 64-dimensional decoupled rotary key, 61 layers, 2 bytes: $576 \times 61 \times 2 = 70{,}272$ bytes ≈ 69 KB. At 100k tokens: 52 GB against 7 GB.

The panel's third point distinguishes compression from sparsity. MLA (slide 15) stores a low-rank latent per token instead of the full keys and values of 128 heads, and reconstructs them on the fly; every token's latent is still read by every query. That is a smaller cache, not a sparser read. DeepSeek-V3.2 added the sparse read nine months later (slide 16).

Terms

H800 The export-market variant of the H100 used by DeepSeek; same compute, reduced interconnect bandwidth.

GPU-hour One GPU running for one hour; 2.788M H800 GPU-hours is about two months on 2,048 GPUs.

multi-head latent attention (MLA) DeepSeek's attention variant that caches a single low-rank latent vector per token (512 wide in V3) plus a small rotary key, and projects keys and values from it when needed. Defined on slide 15.

FP8 An 8-bit floating-point format; two variants, E4M3 and E5M2, defined on slide 18.

Intuition

Read the bar chart as three doors into the same room. The dense model in BF16 walks through 810 GB of bookshelf per word. Quantization makes the books a quarter as thick. The MoE walks past most of the shelves. Doing both (which gpt-oss does, slide 30) compounds, and the attention compression keeps the room from filling up with the conversation itself.

3 Weight pruning (slides 10–13)

3.1 Slide 10: Weight pruning: the classics

Remove the weights that matter least, retrain, repeat. Most of a trained network is slack; the last 10% is not.

What the slide shows A weight matrix shaded by magnitude with the half below a threshold blanked out; the three-step loop *train dense* → *prune* → *retrain* with an arrow back to prune; and a sketch of quality kept against fraction of weights removed, with two curves: with retraining, quality holds past 80–90% removal and then collapses; one-shot without retraining, it falls much earlier. The panel lists the three classical criteria for which weights to remove.

Details *Magnitude pruning.* The simplest criterion is $|w|$: remove the weights with the smallest absolute values. Han, Pool, Tran and Dally (2015) applied it iteratively to AlexNet and VGG-16, pruning a fraction, retraining to recover, and repeating; they reduced AlexNet from 61M to 6.7M parameters (9×) and VGG-16 from 138M to 10.3M (13×) with no loss of accuracy. The retraining is essential: it lets the surviving weights compensate for the removed ones. Done in one shot, without retraining, the same criterion collapses early, as the dashed curve shows.

Second-order criteria. Removing weight w_i changes the loss by an amount that a Taylor expansion around a trained (zero-gradient) minimum gives as

$$\delta\mathscr{L} \approx \tfrac{1}{2}\,\delta w^\top H\,\delta w, \quad \text{with } H \text{ the Hessian of the loss.}$$

Optimal Brain Damage (LeCun, Denker and Solla, 1989) keeps only the diagonal, so $\delta\mathscr{L}_i \approx \frac{1}{2}H_{ii}w_i^2$: a weight is unimportant if it is small *and* the loss is flat in its direction. Optimal Brain Surgeon (Hassibi and Stork, 1993) uses the full inverse Hessian. Removing w_i while adjusting all other weights optimally costs

$$\delta\mathscr{L}_i = \frac{w_i^2}{2\,[H^{-1}]_{ii}}, \qquad \text{with the compensating update} \qquad \delta w = -\frac{w_i}{[H^{-1}]_{ii}}\,H^{-1}e_i,$$

where e_i is the unit vector for coordinate i. OBS is exact to second order and includes the compensation that magnitude pruning gets only from retraining; it was impractical for large networks until SparseGPT (slide 12) restricted it to one layer at a time.

The lottery ticket hypothesis (Frankle and Carbin, 2019). After iterative magnitude pruning, the surviving subnetwork (10–20% of the original in their experiments) can be trained *from its original initialization* to the full network's accuracy, while a random subnetwork of the same size cannot. The reading is that dense training is, among other things, a search for a good sparse subnetwork, and that the mask found is what matters. The result holds cleanly for small networks; for large ones it requires rewinding to an early checkpoint rather than to initialization.

The cliff. The curve's shape is the thing to remember. Trained networks are heavily over-parameterized, and a large fraction of weights can go with negligible effect; past some sparsity the remaining weights can no longer represent the function and quality falls off a cliff. Where the cliff sits depends on the criterion, on whether there is retraining, and on the structure imposed on the zeros (next slide). For large language models in one shot, it is around 50–60% unstructured.

Terms

magnitude pruning Removing the weights of smallest absolute value. The simplest and still a strong baseline when followed by retraining.

iterative pruning Pruning in several rounds with retraining in between, so that each round removes a fraction and the network adapts before the next.

one-shot pruning Pruning once, to the final sparsity, with no retraining (though possibly with a local reconstruction step, as in SparseGPT).

Hessian The matrix of second derivatives of the loss with respect to the weights; it describes the curvature of the loss surface. At a minimum, the loss change from a small perturbation is a quadratic form in it.

Optimal Brain Damage (OBD) Pruning by the diagonal second-order saliency $\frac{1}{2}H_{ii}w_i^2$ (LeCun et al. 1989).

Optimal Brain Surgeon (OBS) Pruning by the full second-order saliency $w_i^2/(2[H^{-1}]_{ii})$ with an optimal compensating update of the other weights (Hassibi and Stork 1993).

saliency A score for how much removing a weight would hurt; magnitude, OBD and OBS are three saliencies.

lottery ticket hypothesis The conjecture, with evidence, that a dense network contains a sparse subnetwork which, trained alone from the same initialization, reaches the dense network's accuracy.

Intuition

A trained network is like a manuscript with far more words than it needs; most can be struck out and the meaning survives, provided the author is allowed a pass to smooth the prose afterwards (retraining). The second-order criteria are the editor's rule for which words to strike: not just the short ones, but the ones whose removal the surrounding sentence can absorb.

3.2 Slide 11: Structure decides the speedup

A zero only saves time if the hardware can skip it. Unstructured zeros rarely do on a GPU; 2:4 and whole blocks do.

What the slide shows Three 6×12 weight grids. *Unstructured*: zeros anywhere. *Semi-structured 2:4*: within every group of four consecutive entries along a row, exactly two are zero; the groups are outlined. *Structured*: entire columns removed. Below each, what runs it and what it buys; a table summarizes realistic sparsity at small loss and the resulting speedup for LLMs.

Details The arithmetic of a matrix multiply on a GPU is done by tensor cores on dense tiles. A zero inside a dense tile costs exactly as much as a nonzero. To save time, the zeros must be arranged so that whole tiles, or whole fixed fractions of tiles, can be skipped, and the matrix must be stored in a compressed form that the hardware can read.

Unstructured sparsity has the best quality per removed weight, because every weight competes on its own saliency, but it is the hardest to accelerate. General sparse matrix kernels (CSR and friends) pay for indexing and irregular memory access, and on GPUs they beat dense kernels only at very high sparsity, around 90% and above. At the 50–60% that LLMs tolerate, an unstructured-sparse checkpoint is smaller on disk and in memory (the index costs bits too) but no faster to run. There are exceptions on CPUs and on purpose-built hardware, and research kernels that exploit sparsity in the memory-bound decode regime, but on a stock GPU the honest statement is "memory, not time".

2:4 semi-structured sparsity (Mishra et al. 2021) is a pattern designed for the hardware: in every group of four consecutive weights along the reduction dimension, two are zero. The matrix is stored compressed to

half size plus a 2-bit index per kept element saying which two of the four positions it occupies. Ampere and later tensor cores read the compressed form and the index and run the multiply at twice the dense rate. Every NVIDIA datasheet since A100 quotes a "with sparsity" peak that is twice the dense one; it refers to this pattern and nothing else. The recipe is to train dense, prune to 2:4 by magnitude within each group of four, and retrain; the pattern is restrictive enough that LLMs lose some quality at 2:4 compared to unstructured 50% (SparseGPT reports a WikiText-2 perplexity increase of about 0.4 for OPT-175B at 2:4), and the end-to-end gain is well below $2\times$ because attention, normalization and the memory traffic are unaffected.

Structured pruning removes whole units: neurons of an MLP (columns of W_{up} and rows of W_{down}), attention heads, embedding channels, or entire layers. The result is simply a smaller dense model, which runs faster on any hardware with no special kernels, and it is the only kind of pruning that reduces depth (latency) and the KV cache (fewer heads or layers). It costs the most quality per removed parameter, because a whole unit must go even if some of its weights were important, and in practice it needs distillation from the parent to recover (slide 13).

Terms

tensor core The matrix-multiply unit of an NVIDIA GPU (AMD: matrix core). It multiplies small dense tiles in a fixed set of formats; the formats it supports define which precisions are fast (slide 21).

2:4 sparsity The semi-structured pattern in which two of every four consecutive weights along a row are zero; supported natively by NVIDIA tensor cores from Ampere on, at twice the dense throughput.

compressed sparse format A storage layout that keeps only nonzeros plus indices; CSR for general sparsity, a 2-bit-per-element index for 2:4.

structured pruning Removing whole neurons, heads, channels or layers, producing a smaller dense network.

semi-structured sparsity A fixed ratio of zeros within small fixed-size groups (such as 2:4); between unstructured and structured in both flexibility and hardware friendliness.

"with sparsity" peak The doubled tensor-core throughput quoted in datasheets, attainable only for 2:4-sparse weights.

Intuition

The hardware does not see zeros; it sees tiles. The question to ask of any pruning result is "what shape are the holes?" Random holes shrink the file; 2:4 holes let the tensor core skip half its work; whole-column holes make the matrix smaller for everyone. Quality per hole runs the other way. Every pruning paper is a point on that trade-off.

3.3 Slide 12: One-shot pruning at LLM scale: SparseGPT and Wanda

No retraining: prune one layer at a time so that its outputs on a few calibration samples barely change.

What the slide shows A pipeline: a calibration set (128 samples of 2k tokens) feeds layer ℓ with inputs X and weights W; the objective is to choose a sparse $\hat{W}$ minimizing $\|WX - \hat{W}X\|_2^2$. Text on SparseGPT and Wanda; a sketch of perplexity against unstructured sparsity, flat to 50% for both methods and rising after, with magnitude pruning diverging early.

Details *Layer-wise reconstruction.* Retraining a 175B-parameter model after pruning is not an option for most practitioners. The alternative is to prune each linear layer separately so that its *output* on representative inputs is preserved: for layer ℓ with input activations X (collected by running a few hundred calibration

sequences through the model) and weights W, find a $\hat{W}$ with the required zeros that minimizes $\|WX - \hat{W}X\|_2^2$. Layers are processed in order, each seeing the outputs of the already-pruned layers before it, so that errors do not compound unseen. The objective is quadratic in $\hat{W}$, its Hessian is $H = XX^\top$ (times 2, by convention), and it needs no labels, no gradients and no backward pass.

SparseGPT (Frantar and Alistarh, 2023) is OBS applied to this layer-wise problem, with two tricks that make it tractable for matrices with tens of thousands of columns. The rows of W share the same input X and hence the same Hessian, so one inverse Hessian serves all rows. Columns are processed left to right; when a column's entries are pruned, the OBS update distributes their error onto the columns not yet processed, using the inverse Hessian of the remaining columns, which is obtained from a single Cholesky factorization rather than recomputed. The mask can be chosen adaptively per block of columns. The result: OPT-175B and BLOOM-176B pruned to 50–60% unstructured sparsity, or to 2:4, in under 4.5 hours on one A100, with a small perplexity increase; the larger the model, the smaller the increase.

Wanda (Sun et al. 2023; "pruning by weights and activations") drops the reconstruction entirely. Its saliency is

$$S_{ij} = |W_{ij}| \cdot \|X_j\|_2,$$

the weight's magnitude times the norm of the input feature it multiplies, compared within each output row. No weight is updated. It matches SparseGPT closely at 50% and runs in minutes. The reason it works, and plain magnitude pruning does not, is the outlier phenomenon of slide 20: a few input channels of an LLM carry values tens of times larger than the rest, so a small weight on an outlier channel contributes more to the output than a large weight on an ordinary one. Magnitude alone misranks these; the activation norm corrects it.

What one-shot pruning does not give you. At 50% unstructured, the model runs no faster on a GPU (slide 11); SparseGPT's 2:4 variant is the one that does. Both methods depend on the calibration set, and a domain shift between calibration and use shows up as quality loss.

Terms

calibration set A small sample of inputs (SparseGPT: 128 sequences of 2048 tokens from C4) used to collect activations for layer-wise pruning or quantization.

layer-wise reconstruction Pruning or quantizing one linear layer at a time to preserve its outputs on calibration data, rather than the end-to-end loss.

Cholesky factorization Decomposing a symmetric positive-definite matrix as $LL^\top$; SparseGPT uses it to obtain the sequence of inverse Hessians for the remaining columns in one pass.

SparseGPT One-shot OBS-style pruning of LLM layers with a shared inverse Hessian and column-wise error propagation (Frantar and Alistarh 2023).

Wanda One-shot pruning by the score $|W_{ij}| \, \|X_j\|_2$ with no weight updates (Sun et al. 2023).

perplexity exp of the average per-token cross-entropy on held-out text; the standard language-modeling metric. Slide 29 explains why it is a weak guide to quantization damage.

Intuition

Layer-wise reconstruction turns an impossible global problem (keep the model's behavior) into a thousand small local ones (keep each layer's output on a sample). It works because each layer is nearly linear in its weights for a fixed input, and the price is that the method only knows about inputs it has seen. Wanda shows how little machinery is needed once you know where the important weights are: on the channels

> that shout.

3.4 Slide 13: Pruning with distillation, and a law for sparsity

The cheap way to a small model is to cut a big one down and teach it from its parent, not to train from scratch.

What the slide shows A pipeline: parent (15B) → importance scores per neuron, head and layer → prune width, then depth → distill with a KL loss to the parent's logits → child (8B or 4B). Text on Minitron and Sheared LLaMA. A sketch of optimal sparsity against training tokens per parameter, rising, after Frantar et al. (2023).

Details *Structured pruning plus distillation.* The structured route of slide 11 produces a smaller dense model, and the quality it loses is recovered not by ordinary fine-tuning but by *distillation*: training the child to match the parent's output distribution (the KL divergence between the parent's and child's next-token probabilities, often plus losses on intermediate hidden states) on a modest amount of text. The parent's logits carry far more information per token than the one-hot next token, which is why so few tokens suffice.

Minitron (Muralidharan et al. 2024, NVIDIA) pruned Nemotron-4 15B to 8B and 4B. Importance of each neuron, head and layer was estimated from activations on a small calibration set (no gradients); the model was pruned in width (MLP neurons, attention heads, embedding channels) and in depth (layers); and the child was distilled from the parent. They report up to 40× fewer training tokens per model than training from scratch, with the pruned-and-distilled models matching or beating similarly sized models trained from scratch, and two practical findings: width pruning hurts less than depth pruning at the same parameter reduction, and distillation beats conventional fine-tuning on the same tokens. The recipe was reused for Llama-3.1-Minitron 4B (from Llama 3.1 8B) and for Mistral NeMo 12B to 8B.

Sheared LLaMA (Xia et al. 2023) pruned LLaMA-2 7B to 2.7B and 1.3B. The masks over layers, heads, hidden dimensions and intermediate dimensions were *learned* with a constrained objective (0.4B tokens), and the pruned model was then continued-pretrained for 50B tokens on a mixture whose domain weights were adjusted dynamically to where the child lagged the parent most. The total was about 3% of the compute of training the small model from scratch, and the children outperformed open models of the same size trained from scratch on far more tokens.

A scaling law for sparsity. Frantar et al. (2023), training sparse T5 and ViT models from scratch at many sizes, sparsities and data budgets, fit a joint law in sparsity S, non-zero parameter count N and data D. Sparsity enters as a multiplicative discount on the parameter term, so a sparse model behaves like a smaller dense one, and the *optimal* sparsity for a fixed number of non-zero parameters rises with the amount of training data. The practical reading: sparsity is most worthwhile for models trained far past their compute-optimal token count, which is exactly the over-trained regime in which small open models are now released (a 8B model on 15T tokens). The curve on the slide is a sketch of that monotone relationship, not a fit.

Terms

distillation Training a student model to match a teacher's outputs (its full next-token distribution, sometimes also its hidden states), rather than only the data's labels. The teacher's soft targets carry more information per example.

KL divergence $D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$, the expected extra code length from using q when the truth is p; the usual distillation loss between teacher and student distributions.

width pruning Removing MLP neurons, attention heads or embedding channels, shrinking every layer.

depth pruning Removing whole layers, shrinking the network's length; it reduces latency most but hurts quality most.

importance score An activation-based or gradient-based estimate of how much a unit (neuron, head, layer) contributes; the structured analogue of saliency.

continued pretraining Resuming next-token training of an existing model on more text, as opposed to fine-tuning on a task.

over-trained model A model trained on many more tokens per parameter than the compute-optimal ratio (around 20); the norm for small open models, and the regime in which sparsity and quantization behave differently (slides 13, 25, 29).

Intuition

Training a small model from scratch makes it rediscover everything. Pruning and distilling a big one hands it the answers: the mask says which parts of the parent's circuitry to keep, and the parent's logits say what the output should look like at every token. That is why 3% of the compute can be enough, and why the small models that matter in 2025 are mostly cut down from bigger ones.

4 Activation and attention sparsity (slides 14–17)

4.1 Slide 14: Activation sparsity: most neurons are silent for most tokens

In a ReLU MLP over 90% of the hidden units are zero per input; skip their rows and the layer is almost free.

What the slide shows An MLP: $x \to W_{\text{up}}$ ($d \to 4d$) $\to$ ReLU $\to W_{\text{down}}$ ($4d \to d$). The hidden vector h is drawn as a strip of 24 cells of which three are lit: only those three rows of W_{down} (and columns of W_{up}) are needed. Text on the measurement and on how the sparsity is induced in models whose activations are not ReLU; a panel on where the time goes.

Details *Measured.* Li et al. (2022, "The lazy neuron phenomenon") measured, in trained T5 and ViT models with ReLU MLPs, the fraction of hidden units that are nonzero for a given input: about 3% for T5-Base, 6% for ViT-B/16, and falling as models grow. The sparsity is not imposed; it emerges from training, and it is per input: different tokens light different units, and across a corpus every unit is used.

Why it saves time, and when. A unit that is zero contributes nothing to $W_{\text{down}}h$, so the row of W_{down} that would multiply it need not be read. If the set of live units can be predicted before computing $W_{\text{up}}x$, the columns of W_{up} for dead units need not be read either. At decode, where the MLP is bound by reading its weights (slide 2), skipping 90% of the rows is a 10× saving on the MLP's bytes; at 50% it is 2×. At prefill, the matmuls are dense and batched over many tokens whose live sets differ, so the union of live units is nearly everything and nothing is saved.

Induced sparsity. Modern LLMs use GELU or SiLU (SwiGLU) gates, whose outputs are small but not zero, so the exact sparsity is lost. Three ways to get it back:

- *ReLUfication* (Mirzadeh et al. 2023, "ReLU strikes back"): replace the activation by ReLU and fine-tune; the paper reports up to a threefold reduction in inference computation with little quality loss, and argues that the choice of GELU/SiLU over ReLU was never justified by quality.
- *Thresholding at inference* (TEAL, Liu et al. 2024; CATS, Lee et al. 2024): zero every hidden unit (and, in TEAL, every input to every linear layer) whose magnitude is below a per-layer threshold chosen offline. TEAL reaches 40–50% model-wide sparsity on Llama-2 and Llama-3 with small degradation and no training, and reports single-batch decode speedups of 1.53–1.8× at 40–50% sparsity with custom kernels. CATS applies the same idea to the gated MLP and reports being within 1–2% of the base model at 50% sparsity.
- *Contextual sparsity* (Deja Vu, Liu et al. 2023): train a small predictor that, from the input to a layer, predicts which MLP units and which attention heads will matter, and load only those. On OPT-175B they observe up to about 85% sparsity (80% of heads, 95% of MLP units) and report over 2× lower latency than FasterTransformer and over 6× lower than Hugging Face's implementation, at somewhat lower sparsity settings.

Predict or test. Testing whether a unit fires requires computing $W_{\text{up}}x$ first, which is half the layer's cost; predicting the live set from the previous layer's output lets the weight loads be issued early and overlapped with computation. This is the engineering content of Deja Vu and of its successors in CPU-offloaded inference (PowerInfer).

MoE as structured activation sparsity. An expert is a group of hidden units that switch on together; the router decides which groups before the matmul, so the sparsity is known in advance, aligned to large blocks, and batches cleanly across tokens. Activation sparsity is the unstructured, threshold-decided version of the same phenomenon.

Terms

activation sparsity The fraction of hidden-unit activations that are zero (or treated as zero) for a given input.

ReLU $\max(0, x)$; produces exact zeros for negative inputs, hence exact sparsity.

GELU, SiLU, SwiGLU Smooth activations ($x\Phi(x)$, $x\sigma(x)$, and a gated pair) used in most modern LLMs; they produce small values rather than zeros.

ReLUfication Replacing a model's activation by ReLU and fine-tuning to recover, to obtain exact sparsity.

contextual sparsity Input-dependent sparsity in units and heads, predicted ahead of time by a small side network (Deja Vu).

lazy neuron A hidden unit that is inactive for most inputs; Li et al.'s name for the phenomenon.

TEAL, CATS Training-free activation-sparsity methods based on magnitude thresholds, applied model-wide (TEAL) or to the gated MLP (CATS).

Intuition

A trained MLP is a dictionary of features, and any one token needs only a few entries. ReLU makes the unused entries exactly zero, which the memory system can exploit at decode; smooth activations blur the zeros into small numbers that still have to be read. The MoE designer's move is to organize the dictionary into chapters and ask the router which chapters to open.

4.2 Slide 15: Attention: where the cost is, and the KV cache

Prefill is quadratic in length; decode reads the whole cache per token. Both are attacked by sharing and by sparsity.

What the slide shows Two cost lines, then four head-sharing schemes (MHA, GQA, MQA, MLA) drawn as a row of query heads over a row of what is cached per token; a table of KV-cache bytes per token for Llama 3 8B, 70B, 405B and DeepSeek-V3, with the total at 128k tokens; and four 8×8 attention masks: full causal, sliding window, window plus sink, block-sparse.

Details *Cost.* For a sequence of length L and model width d, computing attention scores and the weighted values for one layer costs about $4L^2d$ FLOPs in prefill (the $QK^\top$ product and the AV product, each L^2d multiply-adds), quadratic in L. At decode, each new token attends to all L previous tokens, which means reading their cached keys and values: per layer, $2\,h_{kv}\,d_h \cdot \text{bits}/8$ bytes per cached token, times L. The cache is written once at prefill and read once per generated token, so at long context it, rather than the weights, dominates the bytes per token.

Head sharing. Multi-head attention caches a key and a value per head. Multi-query attention (Shazeer 2019) shares one key and value across all heads, cutting the cache by the number of heads; grouped-query attention (Ainslie et al. 2023) shares them within groups, a middle ground that Llama 3 uses with 8 KV heads for 32 to 128 query heads. Multi-head latent attention (DeepSeek-V2) goes further: keys and values are down-projected into a single latent vector per token (512 wide in V3), which is what is cached; they are up-projected when needed, and because the up-projections are linear they can be folded into the query and output projections, so the attention runs directly on the latents. A separate 64-dimensional key carries the rotary position information, which does not commute with the folding. DeepSeek-V2 reports a 93.3% KV-cache reduction relative to its dense predecessor, with quality above MHA rather than below it.

The table (derived from the published configurations; KB = 1024 bytes):

model	layers	KV heads	d_h	KB per token (BF16)	at 128k tokens
Llama 3 8B	32	8	128	128	17.2 GB
Llama 3 70B	80	8	128	320	42.9 GB
Llama 3.1 405B	126	8	128	504	67.6 GB
DeepSeek-V3 (MLA)	61	latent 512 + 64	–	69	9.2 GB

The formula is $2 \times \text{layers} \times h_{kv} \times d_h \times 2$ bytes; for MLA it is $\text{layers} \times (512+64) \times 2$. A 70B model at 128k context needs more memory for one sequence's cache than for half its weights at FP8.

Fixed sparse patterns. The masks on the slide are the classical answers to the quadratic cost: strided and local patterns (Sparse Transformer, Child et al. 2019, $O(L\sqrt{L})$); sliding windows with a few global tokens (Longformer, 2020) and random blocks added (BigBird, 2020), both linear in L; Mistral 7B's 4,096-token sliding window, used in alternation with full attention in later models; and StreamingLLM's observation (Xiao et al. 2023) that the first few tokens act as *attention sinks* that absorb attention mass, so keeping four initial tokens plus a sliding window lets a model run on streams of millions of tokens without degrading. These patterns are cheap and need no learning, and they are blind to content: a fact outside the window is invisible.

Terms

multi-head attention (MHA) Attention with h independent heads, each with its own queries, keys and values; the cache holds h keys and values per token.

multi-query attention (MQA) One key and one value head shared by all query heads; cache reduced by h.

grouped-query attention (GQA) Query heads grouped, each group sharing one key/value head; Llama 3 uses 8 groups.

multi-head latent attention (MLA) Keys and values compressed to a shared low-rank latent per token, cached in that form and expanded by linear maps folded into the attention computation; a separate small rotary key handles positions (DeepSeek-V2/V3).

sliding-window attention Each query attends only to the last W tokens; cost linear in L, memory bounded by W.

attention sink A token (typically the first) that receives a large share of attention mass regardless of content; dropping it destabilizes the softmax, so streaming methods keep it.

block-sparse attention Attention computed only on selected blocks of the $L \times L$ score matrix, so that whole tiles can be skipped by the kernel.

rotary position embedding (RoPE) Encoding position by rotating query and key vectors by position-dependent angles; it does not commute with MLA's folding, hence the separate rotary key.

Intuition

Two different costs hide under the word "attention". Prefill pays for the $L \times L$ score matrix; decode pays for reading L cached tokens per step. Head sharing shrinks the cache by a constant; fixed patterns bound what is read by a window. Neither looks at the content, which is what the next slide adds.

4.3 Slide 16: Learned sparse attention: pick the tokens that matter

Score cheaply, attend fully to the top k: the 2025 designs make long context linear in what is read.

What the slide shows DeepSeek Sparse Attention as a pipeline: the query q_t enters a *lightning indexer* (few heads, FP8) that scores all previous tokens; the top $k = 2048$ are kept; full attention runs over those alone. A paragraph contrasts fixed patterns, eviction and selection. Three cards: NSA, MoBA, and training-free eviction methods.

Details *Selection versus eviction versus patterns.* A fixed pattern (slide 15) decides what to read without looking at content. *Eviction* methods look at content once and discard cache entries permanently, which saves memory and reads but is irreversible: a token dropped for one query is gone for all later queries. *Selection* methods keep the whole cache and choose, per query, a subset to attend to; they save reads but not memory, and they need a scoring mechanism cheap enough to run over the entire cache.

DeepSeek Sparse Attention (DSA; DeepSeek-V3.2-Exp, September 2025, and V3.2, December 2025). A small *indexer* with few heads, run in FP8, computes a score between the current query and every cached token; the top $k = 2048$ tokens by score are selected, and the main attention (MLA) runs over those only. The indexer is $O(L)$ per query but with tiny constants; the attention is $O(k)$. The model was obtained from V3.1-Terminus by continued pretraining: first a dense warm-up in which the indexer is trained to imitate the full attention's distribution, then sparse training. At 128k context the attention reads 2048 of 131,072 tokens, a 64× reduction in cache traffic per query (slide 17), while the cache itself is unchanged.

Native Sparse Attention (NSA; Yuan et al. 2025, DeepSeek). Three branches, summed with learned gates: *compressed* attention over block summaries of the past; *selected* attention over the top-n blocks ranked by the compressed scores; and a *sliding window* over recent tokens. The kernels are written for the block structure, and the model is trained from scratch with the sparse pattern, so the gradients see it. On 64k sequences the paper reports measured kernel speedups of 9.0× forward and 6.0× backward over FlashAttention-2, and an expected decode speedup of up to 11.6× from the reduction in memory traffic.

MoBA (Lu et al. 2025, Moonshot). The cache is divided into blocks; each query is routed to its top-k blocks by a score against each block's mean key, exactly as an MoE router sends tokens to experts ("mixture of block attention"). The method can be switched between full and sparse attention at inference and is deployed in Kimi's long-context models.

Eviction and compression, no training. H2O (Zhang et al. 2023) keeps the *heavy hitters*, tokens that have accumulated the most attention mass, plus recent tokens, with a 20% budget retaining accuracy on their benchmarks. StreamingLLM keeps sinks plus a window. SnapKV (Li et al. 2024) compresses the prompt's cache before generation by keeping, per head, the positions that the last few prompt tokens attend to most. Quest (Tang et al. 2024) keeps everything but, per query, selects pages of the cache by a bound on their possible scores, so it is a selection method without a learned indexer.

Where the gain is. Below about 8k tokens attention is a small part of the step next to the MLPs, and sparse attention buys little. The gain is at decode on long contexts, where the cache read dominates, and in long-context prefill, where the L^2 term dominates. DeepSeek-V4 (2026, †) generalizes the design with compressed sparse attention for million-token windows.

Terms

eviction Permanently removing entries from the KV cache by a rule (attention mass, recency, structure); saves memory and reads, irreversible.

selection Choosing, per query, a subset of the cache to attend to, keeping the rest; saves reads, not memory.

indexer A small network (DSA's *lightning indexer*) that scores every cached token against the query cheaply enough to run over the whole cache.

heavy hitter A cached token that has received a large cumulative attention mass; H2O's criterion for what to keep.

native sparse attention (NSA) DeepSeek's trainable sparse attention with compressed, selected and sliding-window branches and hardware-aligned kernels.

MoBA Mixture of block attention: routing each query to the top-k blocks of the cache by a block-level score.

DeepSeek Sparse Attention (DSA) Indexer-selected top-2048 attention on top of MLA, introduced in DeepSeek-V3.2-Exp.

block A contiguous group of cached tokens treated as a unit by block-sparse methods, so that the kernel can skip whole tiles.

Intuition

The whole design space is one question: how much is it worth to look before you read? Fixed patterns never look. Eviction looks once and burns the bridge. Selection looks every time with a cheap glance, then reads carefully only where the glance pointed. The 2025 result is that the glance can be made cheap enough, and trained well enough, to pay for itself at long context.

4.4 Slide 17: The KV cache is where sparsity and precision meet

Fewer heads, fewer bits, fewer tokens: the three multiply, and the cache at 128k goes from 43 GB to about 2.

What the slide shows A bar chart of the KV cache for one 128k-token sequence of Llama 3 70B: BF16, 42.9 GB; FP8, 21.5; 4-bit, 10.7; 4-bit with 20% of tokens kept, 2.1; and for DeepSeek-V3 with MLA, 9.2 GB at BF16 and 4.6 at FP8. A note distinguishes eviction from selection in what they shrink. Three cards: precision in the cache, sparsity in the cache, and what to expect.

Details The cache is a product of four factors, layers $\times$ heads $\times$ tokens $\times$ bytes, and three of them are attackable.

- *Heads*: GQA with 8 KV heads cuts an MHA cache by the number of query heads per group ($8\times$ for Llama 3 70B's 64 heads); MLA's 576-wide latent is about $7\times$ smaller than Llama's $2 \times 8 \times 128 = 2048$ values per layer.
- *Bytes*: FP8 keys and values with per-head or per-token scales are standard in vLLM and TensorRT-LLM and nearly lossless. Keys are harder than values because they carry the outlier channels (slide 20); KIVI (Liu et al. 2024) quantizes keys per channel and values per token and reaches 2 bits with small loss, and QServe's W4A8KV4 is the integer version at 4 bits.
- *Tokens*: eviction (H2O, SnapKV) keeps a fraction of the tokens and shrinks both the memory and the reads; selection (DSA, NSA, Quest) keeps every token in memory and shrinks only the reads. At $k = 2048$, DeepSeek-V3.2 reads about 2048×69 KB ≈ 140 MB of latent per decoding step from a 9.2 GB cache (derived).

The factors multiply: 4-bit storage at one fifth of the tokens is a $20\times$ reduction in both memory and traffic against BF16 with everything kept.

What to expect. GQA plus FP8 covers most serving today. Past 128k tokens the cache dominates the bytes per token even at FP8, which is why the long-context models of 2025–26 add selection (DSA) or latent compression (MLA), and why KV quantization below 8 bits is an active area. Quality must be checked on long-context tasks (RULER, needle-in-a-haystack with reasoning), because perplexity on short text does not see what the cache lost.

Terms

KV quantization Storing cached keys and values in a low-bit format with per-token or per-channel scales; FP8 routine, 4-bit and 2-bit with care.

KIVI A 2-bit KV-cache quantization with per-channel keys and per-token values (Liu et al. 2024).

per-channel / per-token scaling Sharing a scale factor across a feature dimension (channel) or across a position (token); keys want per-channel because their outliers are channel-aligned.

RULER A long-context benchmark (Hsieh et al. 2024) with thirteen tasks in four categories; most models fall short of their claimed context lengths on it.

Intuition

Think of the cache as a bill with three multiplicative line items. Architecture (GQA, MLA) is paid once, at design time. Precision is paid at serving time and is nearly free down to 8 bits. Token selection is the newest and the only one that scales with the length of the conversation. A model that uses all three is what “million-token context” means in practice.

5 Number formats (slides 18–21)

5.1 Slide 18: Anatomy of a number

Exponent bits buy range, mantissa bits buy precision; every format is a different split of a fixed budget.

What the slide shows Bit layouts, one row per format, with the sign bit in black, exponent bits in purple and mantissa bits in green: FP32 (1/8/23), TF32 (1/8/10), FP16 (1/5/10), BF16 (1/8/7), FP8 E4M3 (1/4/3), FP8 E5M2 (1/5/2), FP4 E2M1 (1/2/1), and the integers INT8 and INT4 in grey. Each row has its largest value and its relative step ε. The panel reads the table.

Details A binary floating-point number with E exponent bits and M mantissa bits represents

$$x = (-1)^s \cdot 2^{e-\text{bias}} \cdot \left(1 + \frac{m}{2^M}\right), \quad e \in [1, 2^E - 2] \text{ for normal numbers}, \quad \text{bias} = 2^{E-1} - 1,$$

with the hidden leading 1 giving $M+1$ bits of significand. The exponent sets the *range*: 2^E binades (factors of two) from the smallest normal to the largest value. The mantissa sets the *precision*: the relative gap between neighbouring representable numbers is $\varepsilon = 2^{-M}$, so a value is known to about $M+1$ binary digits, $0.3(M+1)$ decimal digits. The two trade off within a fixed bit budget, and that trade-off is the whole story of the table.

format	E	M	bits	largest	smallest normal	$\varepsilon = 2^{-M}$
FP32	8	23	32	$3.4 \cdot 10^{38}$	$1.2 \cdot 10^{-38}$	$1.2 \cdot 10^{-7}$
TF32	8	10	19	$3.4 \cdot 10^{38}$	$1.2 \cdot 10^{-38}$	$9.8 \cdot 10^{-4}$
FP16	5	10	16	65,504	$6.1 \cdot 10^{-5}$	$9.8 \cdot 10^{-4}$
BF16	8	7	16	$3.4 \cdot 10^{38}$	$1.2 \cdot 10^{-38}$	$7.8 \cdot 10^{-3}$
FP8 E4M3	4	3	8	448	$2^{-6} = 0.0156$	0.125
FP8 E5M2	5	2	8	57,344	$2^{-14} = 6.1 \cdot 10^{-5}$	0.25
FP4 E2M1	2	1	4	6	1	0.5

Subnormal numbers extend the bottom of the range with reduced precision (E4M3 down to 2^{-9}, E5M2 to 2^{-16}). The FP8 formats follow Micikevicius et al. (2022): E5M2 keeps IEEE conventions (infinities, several NaNs); E4M3 gives up infinities and all but one NaN pattern to gain one more value at the top (448 rather than 240). E2M1's eight positive magnitudes are $0, 0.5, 1, 1.5, 2, 3, 4, 6$.

Reading the table. Eight exponent bits (FP32, BF16, TF32) reach $10^{\pm 38}$, which is why BF16 and TF32 can replace FP32 in training without the loss scaling that FP16 needs (slide 22). Five bits (FP16, E5M2) reach 6×10^4, enough for activations and weights but not for the smallest gradients. Four bits (E4M3) reach 448 and need a scale factor to be useful at all; two bits (E2M1) reach 6 and need a scale per small block. Range is what gradients need, because they span many orders of magnitude; precision is what weights and activations need, because their relative values carry the information. Below 8 bits the per-number precision is so coarse ($\varepsilon = 0.5$ for E2M1) that the format only works as a code within a block that shares a scale (next slide).

Integers have no exponent: INT8 represents $-128..127$ in uniform steps, INT4 represents $-8..7$. Their precision is absolute, not relative, so the scale factor outside them does all the range work.

Terms

exponent The bits of a float that set the power of two; they determine the dynamic range.

mantissa (significand) The bits that set the value within a binade; they determine the relative precision.

binade The interval $[2^k, 2^{k+1})$; each exponent value is one binade, within which the representable

numbers are equally spaced.

machine epsilon $\varepsilon = 2^{-M}$, the relative spacing of representable numbers, or equivalently the largest relative rounding error is $\varepsilon/2$.

subnormal Numbers below the smallest normal, represented without the hidden leading 1; they fill the gap to zero at reduced precision.

TF32 NVIDIA's 19-bit format (8 exponent, 10 mantissa bits) used by tensor cores for FP32 inputs since Ampere; same range as FP32, FP16's precision.

BF16 "Brain float": FP32 with the mantissa truncated to 7 bits; same range, coarser precision.

E4M3, E5M2 The two FP8 formats: 4 exponent and 3 mantissa bits (range to 448), or 5 and 2 (range to 57,344).

E2M1 The FP4 element format: 2 exponent and 1 mantissa bit; values $\pm\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$.

INT8, INT4 Signed integers with 8 or 4 bits; uniform spacing; always used with an external scale factor.

Intuition

A float is scientific notation in base two. The exponent is the "$\times 10^k$" and buys you the ability to write both the mass of the sun and the mass of an electron; the mantissa is the digits in front and buys you the ability to tell 1.000 from 1.001. Cutting a format down means deciding which of the two you can afford to lose, and the answer is different for weights (precision) and gradients (range).

5.2 Slide 19: Scale factors: where the lost bits go

A low-precision number is a code times a scale. The question is how many numbers share one scale.

What the slide shows Two number lines, both scaled to a maximum of 6: the eight positive codes of E2M1 (dense near zero, gaps doubling), and eight evenly spaced INT4 codes. Four small matrices showing which entries share a scale: per tensor, per channel, per token, per group or block. The panel lists the block formats and their real bits per number, and the noise model.

Details *Codes and scales.* A quantized value is $x \approx s \cdot q$ with q a code from the format's set and s a scale chosen so that the largest value in the group fits: $s = \max|x|/q_{\max}$ (absmax scaling). Everything about a quantization scheme is then: which set of codes, and who shares s.

Floats versus integers. E2M1's codes are concentrated near zero, where most weights of a trained network live (they are roughly Gaussian), and spaced ever more widely toward the maximum, so the *relative* error is bounded ($\leq 25\%$) for every value. INT4's codes are evenly spaced, so the *absolute* error is bounded ($\leq$ half a step) and large values are represented accurately while small ones are crushed. Which is better depends on the distribution and on whether the large values (the outliers of slide 20) are what must be preserved.

Granularity of the scale. One scale per *tensor* is cheapest and worst: a single outlier sets the scale for millions of entries. One scale per *output channel* of a weight matrix (a row of W) is the classical choice for weights. One scale per *token* (a row of the activation matrix) is the choice for activations, because their outliers live in fixed channels (slide 20): a per-channel activation scale cannot be used inside a matmul (it would need to be applied per column of X, which is the reduction dimension), so per-token it is, and the outlier channels have to be dealt with otherwise. One scale per *group* of 32–128 consecutive weights along the input dimension (GPTQ, AWQ use 128) limits the damage any one large value can do. Block formats take this to the hardware: one scale per 32 (MX) or 16 (NVFP4) elements, with the tensor core applying the scales itself.

Block formats and honest bit counts.

- *MX* (Open Compute Project Microscaling specification v1.0, 2023; Rouhani et al. 2023): a block of 32 elements shares one 8-bit power-of-two scale (E8M0). MXFP8 has E4M3 or E5M2 elements, $8+8/32=8.25$ bits per number; MXFP6 has E2M3 or E3M2; MXFP4 has E2M1, 4.25 bits; MXINT8 has 8-bit integers. Blackwell tensor cores run MXFP8, MXFP6 and MXFP4 natively.
- *NVFP4* (NVIDIA, 2025): E2M1 elements, blocks of 16 with an E4M3 scale (not a power of two, so it fits the block's maximum more tightly), plus one FP32 scale per tensor: $4+8/16=4.5$ bits per number. The finer block and the non-power-of-two scale are the reasons NVIDIA gives for its better accuracy than MXFP4 at the same element format.
- *INT4 with group 128* (GPTQ, AWQ): a 16-bit scale and often a 4-bit zero point per 128 weights: $4+(16+4)/128\approx 4.16$ bits.
- *NF4* (QLoRA, Dettmers et al. 2023): sixteen levels placed at the quantiles of a standard normal rather than evenly, so that each code is used equally often for Gaussian weights; with *double quantization* (the per-block scales themselves quantized to 8 bits), about 4.1 bits per weight.

Noise. Rounding to a grid of step s adds an error that is approximately uniform on $[-s/2, s/2]$, with variance $s^2/12$. Each bit removed doubles s and quadruples the error variance: 6 dB of signal-to-noise ratio per bit. The only way to remove bits without paying the 6 dB is to keep s small by keeping the blocks small, so that the maximum that sets s is a local maximum, not a global one.

Terms

scale factor The multiplier s that maps a low-precision code back to a value; chosen per tensor, channel, token, group or block.

absmax scaling Setting the scale from the largest absolute value in the group so that it maps to the largest code.

zero point An offset added to integer codes so that an asymmetric range (e.g. activations after ReLU) is used fully; makes the quantization asymmetric.

group quantization Sharing a scale among a fixed number (32–128) of consecutive weights along the input dimension.

block floating point A format in which a block of elements shares one exponent (scale); MX and NVFP4 are block floating-point formats with the scale stored separately.

MX (Microscaling) The OCP block format family: 32 elements per block, E8M0 shared scale, elements FP8/FP6/FP4/INT8.

E8M0 An 8-bit exponent-only scale, a power of two from 2^{-127} to 2^{127}; the MX block scale.

NVFP4 NVIDIA's 4-bit block format: E2M1 elements, 16-element blocks with E4M3 scales, and an FP32 per-tensor scale; 4.5 bits per number.

NF4 NormalFloat 4-bit: a 4-bit code whose sixteen levels are the quantiles of a normal distribution (QLoRA).

double quantization Quantizing the per-block scale factors themselves to a lower precision, saving the overhead bits.

quantization noise The rounding error, modelled as uniform with variance $s^2/12$ for step s.

Intuition

The scale is where the exponent went. A 4-bit code has almost no range of its own, so a block of them borrows one exponent from a shared scale; the block is small enough that its largest member is a fair exponent for the rest. Make the block too big and one giant member ruins the resolution for everyone; make it too small and the scales cost more bits than they save. Sixteen and thirty-two are the numbers the hardware settled on.

5.3 Slide 20: Outliers: the few channels that set everyone's scale

A few hidden dimensions carry values far larger than the rest; every low-precision method works around them.

What the slide shows Three heat maps of an activation matrix (tokens by channels). In the first, two channels are far brighter than all others; in the second, after SmoothQuant's per-channel rescaling, they are moderated; in the third, after a Hadamard rotation, the energy is spread evenly. Text on the three fixes and a panel on why the outliers exist.

Details *The phenomenon.* Dettmers et al. (2022, LLM.int8()) found that in transformer language models above about 6B parameters, a small number of hidden dimensions carry activations with magnitudes tens of times larger than the rest, that these dimensions are the same across tokens and layers, and that they emerge suddenly between 6B and 6.7B parameters in the OPT family. These *outlier features* are systematically important: zeroing them destroys the model, while zeroing a random channel does nothing. Later work (Sun et al. 2024, "Massive activations") showed that a handful of activations at specific tokens (often the first token and delimiters) reach values thousands of times the median and act as fixed biases that steer attention, which is the same mechanism as the attention sinks of slide 15.

Why they are a problem for low precision. Activations are quantized per token (slide 19). A token's scale is set by its largest value, which is the outlier channel; every other channel of that token is then represented with a few codes near zero. INT8 per-token quantization of a 6.7B+ model collapses for this reason, and nothing smaller than 8 bits is possible without treating the outliers specially.

Three fixes.

- *Mixed-precision decomposition* (LLM.int8()): the outlier columns of X (and the corresponding rows of W) are multiplied in FP16; the rest in INT8 with vector-wise scales. With a threshold of 6.0 on the activation magnitude, the number of outlier columns is small, and the method reproduces the FP16 model's quality up to 175B parameters with no degradation.
- *SmoothQuant* (Xiao et al. 2022): migrate the difficulty from activations to weights by an exact rescaling. For input channel j, divide $X_{:,j}$ by s_j and multiply $W_{j,:}$ by s_j (so XW is unchanged), with

  $$s_j = \frac{\max|X_{:,j}|^{\alpha}}{\max|W_{j,:}|^{1-\alpha}}, \qquad \alpha = 0.5 \text{ in general (0.75 for the hardest models).}$$

  The activation outliers shrink by $\sqrt{\text{their size}}$, the weights grow by the same, and both become quantizable to 8 bits with ordinary per-tensor or per-token scales; the result is a plain W8A8 integer matmul with no mixed precision, up to 1.56× faster and half the memory of FP16 on OPT-175B.
- *Rotations* (QuaRot, Ashkboos et al. 2024; SpinQuant, Liu et al. 2024): multiply the activations by an orthogonal matrix H and fold $H^{\top}$ into the following weights: $XW = (XH)(H^{\top}W)$. A Hadamard matrix spreads each outlier across all channels, so the maximum of XH falls by a factor approaching $\sqrt{d}$ while the norms are unchanged (*incoherence*). The same trick is applied inside attention to keys and values,

which is what makes 4-bit KV caches possible. QuaRot uses random Hadamard matrices and reaches 4-bit weights, activations and cache on Llama-2 70B with a WikiText-2 perplexity increase of at most 0.47; SpinQuant learns the rotations on the Stiefel manifold and is one of the two methods Meta used for its Llama 3.2 quantized releases. On gradients, the same rotation is what lets FP4 training work (slide 24).

Why they exist. The outlier channels appear to be the network's way of implementing constants: a few dimensions pushed to huge values act as biases that the layer norm and softmax then rely on (a large constant in the pre-norm residual pins the norm's scale; a large key dimension gives the attention sink). They are a feature of trained transformers, not noise, which is why removing them breaks the model and why Wanda's pruning criterion (slide 12) weighs weights by the activation norm.

Terms

outlier feature A hidden dimension whose activations are tens to thousands of times larger than typical, present in the same dimensions across tokens and layers in models above about 6B parameters.

mixed-precision decomposition Computing the outlier columns of a matmul in high precision and the rest in low precision (LLM.int8()).

SmoothQuant An exact per-channel rescaling that moves activation outliers into the weights so that both quantize to 8 bits.

migration strength (alpha) SmoothQuant's exponent controlling how much of the difficulty is moved to the weights; 0.5 splits it evenly.

Hadamard matrix An orthogonal matrix with entries $\pm 1/\sqrt{d}$; multiplying by it mixes every coordinate into every other, flattening outliers. Fast to apply ($O(d \log d)$).

incoherence A property of a matrix whose entries are all of similar magnitude relative to its norm; rotations make weights and activations incoherent so that uniform quantization works.

QuaRot, SpinQuant Rotation-based quantization methods using random (QuaRot) or learned (SpinQuant) orthogonal matrices folded into the weights.

massive activations A few activations at specific tokens with magnitudes thousands of times the median, acting as fixed biases (Sun et al. 2024).

Intuition

Two people in a room of a hundred are shouting; a microphone set to not clip on them records everyone else as a whisper. You can give the shouters their own microphone (decomposition), ask them to speak more softly and turn the gain up on the recording (SmoothQuant), or seat everyone in a mixing chamber so that each microphone hears a blend (rotation). All three are in use; the third is the one that scales to four bits.

5.4 Slide 21: What the tensor cores run, by generation

Each generation halves the smallest native format and doubles its throughput; bandwidth grows slower than FLOPs.

What the slide shows A table by GPU generation: the native formats introduced, dense tensor-core peaks in BF16, FP8 and FP4, memory capacity, bandwidth, and the BF16 ridge point. A100, H100 SXM, B200, AMD MI300X and MI355X, and NVIDIA's Rubin (†). Three cards: format support is the lever; FLOPs outrun bytes; accumulation is the fine print.

Details The table, with the numbers' provenance.

GPU	new native formats	BF16	FP8	FP4	HBM	TB/s	ridge
		dense TFLOP/s			GB		FLOP/B
A100 (2020)	TF32, BF16, INT8; 2:4	312	–	–	80	2.0	156
H100 SXM (2022)	FP8 E4M3/E5M2	989	1,979	–	80	3.35	295
B200 (2024)	FP4 (NVFP4, MXFP4), FP6	2,250	4,500	9,000	180	8.0	281
MI300X (2023)	FP8	1,307	2,615	–	192	5.3	247
MI355X (2025)	FP4, FP6	2,520	5,030	10,070	288	8.0	315
Rubin (2026)†	NVFP4 training	–	–	35,000	288	22	–

A100 and H100 figures are from NVIDIA's datasheets, which list the H100 numbers "with sparsity" (1,979 BF16, 3,958 FP8) so the dense values are half. NVIDIA publishes B200 numbers only as 8-GPU HGX totals (FP4 72 PFLOP/s dense, FP8 and BF16 given with sparsity), so the per-GPU column is derived by dividing by eight; older 2024 materials quoted 192 GB and a 20 PFLOP/s sparse FP4 figure for a different configuration. MI300X and MI355X are from AMD's datasheets, which list dense and sparse separately. Rubin's numbers are from NVIDIA's 2026 announcement (35 PFLOP/s NVFP4 for training, 50 with sparsity for inference, 288 GB HBM4, up to 22 TB/s) and have not been measured independently. The ridge is BF16 peak over bandwidth, derived.

Three observations.

- A format pays only when the tensor core multiplies it natively. Each arrival (INT8 on Ampere, FP8 on Hopper, FP4 on Blackwell) moved the default serving precision within about a year, because the kernels followed the hardware.
- From A100 to B200, dense BF16 throughput rose about 7× while bandwidth rose 4×. The ridge drifts upward, so small-batch decode becomes more memory-bound with every generation, and the value of fewer bits per weight rises with it. Rubin's 22 TB/s is a partial correction.
- Products in a low format are accumulated in a wider one, and how wide is part of the specification that vendors do not always publish. DeepSeek-V3's team measured that FP8 matrix multiplication on H800 retains only about 14 bits of accumulation precision in the tensor core, enough to bias long reductions, and moved partial sums to FP32 registers every 128 terms (slide 23). This is DeepSeek's empirical finding, not an NVIDIA specification.

Terms

native format A format the tensor core multiplies directly, at full rate, without conversion; conversions in software cost more than they save.

dense peak Tensor-core throughput for ordinary (non-2:4-sparse) matrices; half the datasheet headline.

accumulation precision The width of the running sum inside a dot product; products of low-precision inputs must be summed in something wider, and if the accumulator is too narrow, long sums lose low-order terms.

Transformer Engine NVIDIA's library (Hopper onward) that manages FP8 and FP4 scaling, casting and recipes for transformer layers.

HBM3e, HBM4 Generations of high-bandwidth memory; HBM3e on B200 and MI355X (8 TB/s), HBM4 on Rubin.

Intuition

The table is a staircase and a treadmill. The staircase is formats: every generation adds a native format with half the bits and double the rate. The treadmill is bandwidth, which cannot keep up, so the gap between what the chip can compute and what it can feed widens, and the only way to use the new arithmetic at small batch is to send it smaller numbers.

6 Low-precision training (slides 22–24)

6.1 Slide 22: Mixed-precision training: master weights, loss scaling, BF16

Compute in 16 bits, accumulate and update in 32: the recipe since 2017, and why BF16 made it simpler.

What the slide shows The training loop as four boxes: BF16 weights cast each step; the forward pass (BF16 $\times$ BF16 with FP32 accumulation); the backward pass with BF16 gradients; the optimizer in FP32 holding the master weights and Adam's moments; and the cast back. The bytes per parameter are counted: $2+2+4+4+4=16$. Text on the master copy, loss scaling and BF16; a panel on what stays in 32 bits.

Details *The recipe* (Micikevicius et al. 2018). Keep an FP32 *master copy* of the weights; at each step cast it to 16 bits for the forward and backward passes; compute matmuls in 16 bits with FP32 accumulation; apply the optimizer update to the FP32 copy. The memory per parameter is 2 bytes for the 16-bit weights, 2 for the 16-bit gradient, and 12 for the FP32 master weight and Adam's two moments, 16 bytes in all (ZeRO's accounting), plus activations.

Why a master copy. An optimizer step adds ηg to each weight, and ηg is often 10^{-4} of the weight or smaller. In BF16, whose relative step is $2^{-7} \approx 0.8\%$, $w + \eta g$ rounds back to w whenever $|\eta g| < 0.4\%\,|w|$, and training silently stops moving most weights. Accumulating the update in FP32, where the relative step is 10^{-7}, keeps every contribution; the cast to 16 bits for the forward pass loses only what the forward pass could not have used anyway.

Loss scaling. FP16's smallest normal number is 6×10^{-5} and most gradients are smaller, so they underflow to zero. The fix is to multiply the loss by a constant S (which multiplies every gradient by S, pushing them into range), compute the backward pass, then divide the gradients by S before the update. Dynamic loss scaling raises S while nothing overflows and halves it when a gradient hits infinity.

BF16. With FP32's eight exponent bits, BF16 has the same range as FP32 and nothing underflows, so loss scaling disappears; the price is seven mantissa bits, less precision than FP16's ten, which the FP32 master absorbs. BF16 became the default on TPUs and, with Ampere, on GPUs.

What stays in FP32. The matmuls run in the low format with FP32 accumulation. Softmax, layer norm, the residual additions, the loss, and the optimizer state stay in FP32 or in BF16 with care; gradient all-reduce often goes in BF16, and in FP8 in some systems. Memory pressure pushes the other way: DeepSeek-V3 keeps Adam's m and v in BF16, with only the master weights and gradient accumulation in FP32, bringing the 16 bytes per parameter down to 12.

Terms

mixed-precision training Computing the expensive operations in a low-precision format while keeping the weights' master copy and the sensitive reductions in FP32.

master weights The FP32 copy of the parameters that the optimizer updates; the low-precision weights are a cast of it.

loss scaling Multiplying the loss by S to lift gradients above FP16's underflow threshold, and dividing by S before the update; static or dynamic.

underflow A value too small for the format rounding to zero (or to a subnormal with few significant bits).

Adam moments The optimizer's running averages of the gradient (m) and of its square (v), one of each per parameter, usually in FP32.

ZeRO A family of memory optimizations that partition optimizer state, gradients and weights across data-parallel workers; its paper's accounting gives the 16 bytes per parameter.

all-reduce The collective that sums gradients across data-parallel workers; its precision is a separate choice from the matmuls'.

Intuition

Training is a long sum of tiny corrections. The forward pass can afford to be rough (it only needs to know roughly what the model would say), but the sum of corrections cannot, because a correction too small to register is a correction lost forever. Hence the split: rough arithmetic, exact bookkeeping.

6.2 Slide 23: FP8 training: the scale goes per tensor, then per tile

Eight bits hold a GEMM if each small block has its own scale and the sums are carried in 32.

What the slide shows An activation matrix X with one scale per 1×128 tile along the reduction dimension, a weight matrix W with one scale per 128×128 block, feeding a tensor core (FP8 $\times$ FP8, short sums) whose partial sums are added in FP32 on the CUDA cores every 128 terms. Text on Transformer Engine, DeepSeek-V3's recipe, and NVIDIA's MXFP8 recipe.

Details *Transformer Engine (2022).* NVIDIA's FP8 recipe for Hopper keeps one scale per tensor. The scale is set from the tensor's maximum absolute value, but computing the maximum before the cast would serialize the pipeline, so *delayed scaling* uses a history of recent maxima (default length 1024 steps) and takes the max of the window. The default format is "hybrid": E4M3 for weights and activations in the forward pass, E5M2 for gradients in the backward pass (range matters more for gradients), with FP32 master weights and optimizer. Later versions add *current scaling*, computing the maximum on the fly.

DeepSeek-V3 (2024). The first open model at the 600B scale trained predominantly in FP8, and the clearest public description of what it takes. The choices:

- E4M3 for all FP8 tensors, including gradients: with fine-grained scaling the range of E5M2 is not needed and the precision of E4M3 is.
- *Fine-grained scaling*: activations get one scale per 1×128 tile (one token, 128 channels); weights get one scale per 128×128 block. An outlier spoils only its own tile instead of the whole tensor, which is the per-block principle of slide 19 applied inside a GEMM.
- *Increased accumulation precision*: the team measured that FP8 GEMMs on H800 retain only about 14 bits of accumulation inside the tensor core. The fix is to accumulate $N_C = 128$ products in the tensor core, then move the partial sum to FP32 registers on the CUDA cores and continue; the tile and block scales are applied at that point. The kernel is written so that the promotion overlaps with the next tensor-core instruction.
- What stays higher: the embedding, the output head, the MoE gating, the normalization layers and the attention operators run in BF16 or FP32; the optimizer keeps master weights and gradient accumulation in FP32 and Adam's moments in BF16.
- Result: on a 16B and a 230B MoE trained to about 1T tokens, the relative loss error against a BF16 baseline stays below 0.25%, within training noise.

MXFP8 (NVIDIA, 2025). The block-format version: E4M3 elements with an E8M0 scale per 32, for weights, activations and gradients. The paper's main finding is about the scale: the OCP v1.0 specification's method effectively rounds the scale down, which lets the block's largest element overflow and saturate; rounding the scale *up toward* $+\infty$ avoids this, and with it an 8B model trained on 15T tokens matches BF16 in validation perplexity (within 0.5%) and downstream tasks.

Terms

GEMM General matrix multiply, $C = \alpha AB + \beta C$; the operation the tensor cores run and the one that is quantized.

delayed scaling Choosing an FP8 tensor's scale from a history of recent maxima rather than the current one, so that the cast does not wait for a reduction.

current scaling Computing the scale from the current tensor's maximum.

tile and block scaling DeepSeek-V3's scheme: scales per 1×128 activation tile and per 128×128 weight block.

promotion Moving a partial sum from the tensor core's accumulator to a wider (FP32) register at intervals, to bound the accumulation error.

WGMMA Hopper's warp-group matrix-multiply-accumulate instruction; DeepSeek's interval of 128 corresponds to four of them.

MXFP8 The MX block format with FP8 elements; NVIDIA's 2025 recipe trains in it with round-up scales.

Intuition

FP8 training is BF16 training with two extra habits: give every small block its own ruler, and never let a long sum live in a short register. DeepSeek-V3 is the demonstration that, with those two habits, eight bits are enough for a frontier model; the recipe fits on this slide, and the kernels took a team.

6.3 Slide 24: FP4 training arrives

Four bits need everything the FP8 recipe has, plus unbiased rounding and a rotation before quantizing.

What the slide shows A chevron timeline of training precisions: FP32 (2012), FP16 with loss scaling (2017), BF16 (2019–20), FP8 (Transformer Engine 2022, DeepSeek-V3 2024), FP4 (MXFP4 and NVFP4, 2025). Six cards: 2D block scaling for weights, random Hadamard transforms, stochastic rounding, keeping the sensitive layers, what was shown, and where it stands.

Details At four bits the element format has eight magnitudes and $\varepsilon = 0.5$; nothing on slide 23 is enough by itself. The 2025 recipes (Tseng et al. for MXFP4; NVIDIA for NVFP4; Castro et al., Quartet) add four ingredients.

- *2D block scaling for weights.* In the backward pass the weight matrix is used transposed. If scales are per 1×16 block along one dimension, the transposed matrix's blocks run the other way, and the same weight quantizes to different values in the forward and backward passes, which biases the gradient. A 16×16 block scale makes W and $W^\top$ quantize identically. Activations and gradients keep 1×16 blocks.
- *Random Hadamard transforms.* Before quantizing a gradient (and, in some recipes, activations), rotate it by a random Hadamard matrix (slide 20) so that no single outlier sets a block's scale; the rotation is orthogonal and is undone exactly after the matmul by rotating the other operand. Tseng et al. show that the transform also bounds the variance of stochastic rounding.
- *Stochastic rounding.* Round up or down at random with probability proportional to the distance to each neighbour, so that the expected quantized value equals the true value. Round-to-nearest systematically pushes small gradient components to zero, which at 4 bits is most of them; stochastic rounding keeps the gradient unbiased at the cost of variance.

- *Keep the sensitive layers.* Embeddings, normalization, the output head, and the first and last transformer blocks stay in BF16 or FP8. NVIDIA's NVFP4 pretraining kept 16% of the linear layers in higher precision (the first two and last eight blocks).

What was shown. NVIDIA (2025): a 12B hybrid Mamba-Transformer trained on 10T tokens in NVFP4, with a training-loss gap to the FP8 baseline below 1% through most of training and about 1.5% late, and downstream scores matching (MMLU-Pro 62.6 vs 62.6; some coding tasks slightly below, some math tasks above). Tseng et al. (2025): a near-lossless MXFP4 recipe with stochastic rounding and random Hadamard transforms that computes more than half the training FLOPs in MXFP4, with an estimated backward-pass speedup over FP8 of more than 1.3×. Quartet (Castro et al. 2025): end-to-end FP4 training with Blackwell kernels, presented as competitive with FP16 and FP8.

Where it stands (†). Blackwell runs FP4 natively and the recipes are public. In 2026, NVIDIA's Nemotron 3 Ultra (550B total, 55B active) reports pretraining in NVFP4 on 20T tokens with a loss gap below 0.4% against BF16 segments; DeepSeek-V4's model card states that its MoE expert weights are in FP4 with most other parameters in FP8; Kimi K3's card describes MXFP4-weight, MXFP8-activation quantization-aware training from the supervised fine-tuning stage on. These are the labs' own reports, without the detail of DeepSeek-V3's FP8 description.

Terms

stochastic rounding Rounding to the upper or lower neighbour with probabilities proportional to proximity, so that the rounding is unbiased in expectation.

round-to-nearest Deterministic rounding to the closest representable value; biased toward zero for values below half a step.

random Hadamard transform (RHT) Multiplication by a Hadamard matrix with random sign flips, used to spread outliers before quantization and undone after.

2D block scaling Sharing a scale over a square block (16×16) of a weight matrix so that the matrix and its transpose quantize identically.

MXFP4 The MX block format with E2M1 elements and a power-of-two scale per 32; 4.25 bits per number.

Quartet Castro et al.'s 2025 FP4 training method with kernels for Blackwell.

quantization-aware training (QAT) Training with the quantization applied in the forward pass so that the weights adapt to it; defined fully on slide 28.

Intuition

Each halving of precision has needed one new idea to keep the gradient honest: FP16 needed scaling to stop underflow; BF16 needed the FP32 master to stop the updates vanishing; FP8 needed per-block scales to stop outliers; FP4 needs rounding that is right on average and a rotation that leaves no outlier to be right about. The pattern suggests that the next step, if there is one, will be about the statistics of rounding rather than about the format.

7 Scaling laws for precision (slide 25)

7.1 Slide 25: Scaling laws for precision

Bits are a resource like parameters and tokens: there is a compute-optimal precision, and it is not 16.

What the slide shows Two sketches. Left: loss against training precision in bits at a fixed FLOP budget, falling steeply from 3 bits, flattening, and rising slowly past an optimum near 7–8 bits (after Kumar et al. 2024). Right: zero-shot accuracy against total model bits (parameters times bits) for 3-, 4-, 8- and 16-bit models, with the 4-bit line highest at every budget and 3-bit below (after Dettmers and Zettlemoyer 2023). A panel on post-training quantization and the token-per-parameter effect.

Details *Training precision* (Kumar et al. 2024, "Scaling laws for precision"). The paper's device is an *effective parameter count*: a model with N parameters trained at precision P behaves, in the Chinchilla loss form, like a dense model with

$$N_{\text{eff}}(N,P) = N\left(1 - e^{-P/\gamma}\right), \qquad L(N,D,P) = \frac{A}{N_{\text{eff}}^{\alpha}} + \frac{B}{D^{\beta}} + E,$$

with separate P and γ for weights, activations and the KV cache ($\gamma_w \approx 2.7$ bits for weights in their fits). Lower precision discounts the parameters; the discount is steep below γ and negligible far above it. At a fixed compute budget, FLOPs scale as $N \cdot D \cdot P$ (lower precision is cheaper per parameter), so lowering P lets one buy more N or D; the loss falls until the discount on N_{eff} overtakes the gain, which in their integer-quantization fits happens around 7–8 bits. Two consequences they draw: training at 16 bits is wasteful under this law, and training larger models in lower precision may be compute-optimal. The fits are from 465 runs of models up to 1.7B parameters and 26B tokens, so the constants are not the frontier's, but the shape is the point.

Post-training quantization. The same paper fits the loss *increase* from quantizing a trained model's weights to P_{post} bits:

$$\delta_{\text{PTQ}} \propto \frac{D^{\gamma_D}}{N^{\gamma_N}} e^{-P_{\text{post}}/\gamma_{\text{post}}},$$

increasing in the training tokens per parameter. A model trained on more data packs more information into each weight, and rounding destroys more of it; past some amount of data, additional pretraining tokens make the quantized model *worse*. The over-trained small models that are now the norm (8B on 15T tokens, nearly 2000 tokens per parameter) are therefore the hardest to quantize, which practitioners had noticed and this law explains. Ouyang et al. (2024) make the same point under the title "low-bit quantization favors undertrained LLMs".

Inference precision (Dettmers and Zettlemoyer 2023, "The case for 4-bit precision"). Across more than 35,000 experiments on models from 19M to 176B parameters and several families, with the model size and the bit width both varied, 4-bit weights are almost universally optimal for zero-shot accuracy per total model bit: at any memory budget, a larger model at 4 bits beats a smaller one at 8 or 16, and 3 bits loses. Small block sizes and data types with more quantile-like level placement improve the 4-bit point further.

Floating-point laws. Sun et al. (2025) extend the training law to floating formats with exponent and mantissa bits as separate variables, finding that exponent bits contribute slightly more than mantissa bits at the same total and that the cost-optimal precision lies between 4 and 8 bits.

Terms

effective parameters N_{eff}: the dense-equivalent parameter count of a model trained or stored at reduced precision, $N(1-e^{-P/\gamma})$ in Kumar et al.

Chinchilla loss form $L = A/N^{\alpha} + B/D^{\beta} + E$, the parametric scaling law of Hoffmann et al. (2022), extended here with precision.

compute-optimal precision The number of bits that minimizes loss at a fixed FLOP budget when N, D and P are traded off; about 7–8 in the integer fits.

post-training quantization (PTQ) Quantizing a model after training without further gradient updates; its loss increase is the δ_{PTQ} of the law.

tokens per parameter D/N; about 20 at the Chinchilla optimum, hundreds to thousands for over-trained small models.

total model bits Parameters times bits per parameter; the memory budget axis in Dettmers and Zettlemoyer's comparison.

Intuition

Precision is a third axis of the scaling-law cube, and it has the same shape as the other two: diminishing returns above a scale set by a constant γ of a few bits. Below that scale, bits are parameters; above it, they are waste. The one twist is that data works against precision after the fact: the more a model has learned per weight, the less rounding it can take.

8 Quantization for inference (slides 26–29)

8.1 Slide 26: Post-training quantization I: weights only

Round the weights to 4 bits after training, one layer at a time; decode gets faster because it reads fewer bytes.

What the slide shows GPTQ's column sweep over a weight matrix: columns already quantized to 4 bits on the left, the current column highlighted, and the columns still to come receiving a correction. Text on GPTQ and AWQ; a panel on why weight-only quantization speeds up decoding, the formats found in practice, and what happens below 4 bits.

Details *GPTQ* (Frantar et al. 2022) applies the layer-wise reconstruction of slide 12 to quantization rather than pruning, using the Optimal Brain Quantization formulation (OBQ, Frantar and Alistarh 2022): quantize one weight, then update the remaining weights of the row to compensate, using the inverse Hessian H^{-1} of the layer-wise objective, $H = 2XX^\top$. OBQ's per-weight greedy order is too slow for LLM matrices, so GPTQ makes three changes: quantize whole columns in a fixed (arbitrary) order, since at scale the order barely matters; process columns in lazy batches of 128 so that the updates to the remaining columns are applied as one matrix operation; and obtain the sequence of inverse Hessians from a single Cholesky decomposition, which is also numerically stabler. For column q with quantization error $w_q - \mathrm{quant}(w_q)$, the remaining columns are updated by

$$W_{:,>q} \leftarrow W_{:,>q} - \frac{w_q - \mathrm{quant}(w_q)}{[H^{-1}]_{qq}} [H^{-1}]_{q,>q}.$$

The result: OPT-175B quantized to 3 or 4 bits per weight in about four GPU-hours with small perplexity loss, and end-to-end decode speedups of about 3.25× on an A100 and 4.5× on an A6000 against FP16, from the memory reduction alone.

AWQ (Lin et al. 2023, "activation-aware weight quantization") starts from the observation that not all weights matter equally: the roughly 1% of weight channels that multiply the largest activations account for most of the quantization error, and keeping them in FP16 nearly recovers the full model. Mixed precision is awkward in kernels, so AWQ instead scales those channels up before rounding (multiplying W's input channel j by s_j and dividing X's channel j by s_j, exactly as SmoothQuant does in the other direction): a larger weight is rounded with a smaller relative error. The per-channel scales are found by a grid search on calibration data that minimizes the output error of the layer. No backpropagation, no reconstruction; 4-bit weights in groups of 128; and a serving system (TinyChat) that the paper reports at more than 3× the speed of the FP16 Hugging Face implementation.

Why weight-only quantization is faster, and when. At small batch, decode reads every weight once per token (slide 2). Four-bit weights are a quarter of the bytes of BF16, so the step is up to four times faster; the kernel dequantizes the weights to 16 bits on the fly as they arrive, which costs arithmetic the chip has to spare. Marlin (Frantar et al. 2024) is the reference kernel for this *W4A16* setting and reaches close to the ideal 4× for batches up to 16–32. Past the ridge, the layer is compute-bound, dequantization is overhead, and weight-only quantization gains nothing; large-batch serving needs the activations quantized too (next slide).

Formats in the wild. GPTQ and AWQ checkpoints are standard on the Hugging Face Hub; llama.cpp's *k-quants* offer 2- to 6-bit variants with mixed group sizes for local inference; NF4 is the format of QLoRA fine-tuning. All use per-group scales and cost 4.1–4.6 real bits per weight (slide 19).

Below four bits. The 2024 methods that reach 2–3 bits (QuIP#, Tseng et al.; AQLM, Egiazarian et al.) replace scalar rounding by *vector quantization*: groups of weights are mapped to the nearest entry of a learned codebook (AQLM: additive codebooks; QuIP#: a lattice codebook after a Hadamard incoherence transform),

with fine-tuning of the remaining parameters. AQLM reports being the first Pareto-optimal scheme below 3 bits; the quality cost at 2 bits is substantial and grows with how over-trained the model is (slide 25).

Terms

weight-only quantization Quantizing the weights while leaving activations in 16 bits (W4A16); the kernel dequantizes weights as it reads them.

GPTQ Layer-wise, second-order post-training quantization of LLM weights with column-wise error feedback (Frantar et al. 2022).

OBQ Optimal Brain Quantization: the per-weight greedy quantize-and-compensate method that GPTQ scales up.

AWQ Activation-aware weight quantization: per-channel scaling that protects the salient weights before rounding (Lin et al. 2023).

salient weights The weights whose quantization error affects the output most; in AWQ, those on channels with large activations.

dequantization Converting low-bit codes back to a compute format (BF16) inside the kernel, just before the multiply.

Marlin A W4A16 GEMM kernel that reaches near-ideal 4× memory speedup at small batch (Frantar et al. 2024).

k-quants llama.cpp's family of block quantization formats at 2–6 bits.

vector quantization Quantizing groups of values jointly to entries of a codebook rather than each value to a grid; the basis of the 2-bit methods QuIP# and AQLM.

codebook The finite set of vectors to which groups of weights are snapped in vector quantization.

Intuition

Weight-only quantization is compression for the bookshelf of slide 2: thinner books, same reader. It helps exactly when the reader is waiting on the shelf, which is small-batch decode, and it is irrelevant when the reader is the bottleneck. GPTQ and AWQ are two ways to decide what to round: GPTQ by correcting the neighbours, AWQ by protecting the few that matter.

8.2 Slide 27: Post-training quantization II: activations too

Weight bits help decode; activation bits help prefill and large batches. Hardware decides which pairs run natively.

What the slide shows A grid of weight bits (16, 8, 4, 2) against activation bits (16, 8, 4) with the methods that occupy each cell: W4A16 (GPTQ, AWQ, k-quants, the local default), W8A8 (SmoothQuant and LLM.int8 in INT8; FP8 as the serving default), W4A8 (QServe, Atom), W4A4 (QuaRot, SpinQuant; NVFP4 native on Blackwell), W2A16 (QuIP#, AQLM). A note on which cells are native. The panel explains the two halves of serving.

Details *Two regimes.* Prefill and large-batch decode are compute-bound (slide 2). There the time is set by the tensor-core rate for the format of the *matmul inputs*, which means both weights and activations must be in a low native format: W4A16 gains nothing, W8A8 in FP8 runs at twice BF16's rate on Hopper, W4A4 in NVFP4 at four times on Blackwell. Small-batch decode is bandwidth-bound; weight bits are what count and

activations may stay at 16.

W8A8. INT8 for both requires dealing with activation outliers: SmoothQuant's offline rescaling (slide 20) with calibration, or LLM.int8()'s decomposition. FP8 (E4M3) W8A8 on Hopper and later is simpler: per-token dynamic scales for activations and per-channel scales for weights, computed on the fly, no calibration, and quality indistinguishable from BF16 for most models; it has been the default of production serving stacks (vLLM, TensorRT-LLM, SGLang) since 2024.

W4A8 and W4A4. QServe (Lin et al. 2024) runs 4-bit weights with 8-bit activations and a 4-bit KV cache (W4A8KV4), dequantizing weights to INT8 in registers, and reports 1.2–3.5× over TensorRT-LLM on A100 and L40S depending on the model. Atom (Zhao et al. 2023) is the earlier W4A4 attempt with outlier channels kept in INT8. Four-bit activations in general require the rotations of slide 20: QuaRot and SpinQuant quantize weights, activations and the KV cache to 4 bits with Hadamard or learned rotations folded into the weights. On Blackwell the W4A4 cell becomes native through NVFP4 with its 16-element block scales; NVIDIA reports (†) accuracy close to FP8 on common benchmarks for post-training-quantized models, with the usual caveat that the hard tasks are where the differences appear.

Native cells. A cell is fast only when its pair of formats matches a tensor-core format: INT8×INT8 since Ampere (Turing for INT8 inference), FP8×FP8 since Hopper, FP4×FP4 since Blackwell. Everything else is a memory trick with dequantization in the kernel, useful at small batch and neutral or harmful at large.

The KV cache follows the same split (slide 17): per-token scales, FP8 routinely, 4 bits with care. And one model skips the problem: DeepSeek-V3 ships FP8 weights from training, so its serving precision is its training precision and no PTQ step is needed.

Terms

W8A8, W4A16, W4A8, W4A4 Shorthand for weight bits and activation bits; W4A8KV4 adds the KV-cache bits.

dynamic quantization Computing activation scales at run time per token or per tensor, rather than from a calibration set.

static quantization Fixing activation scales from calibration data in advance; needed for INT8 pipelines that cannot afford the runtime maximum.

QServe A W4A8KV4 serving system with register-level dequantization (Lin et al. 2024).

Atom An early W4A4 method keeping outlier channels in INT8 (Zhao et al. 2023).

serving stack The inference system (vLLM, TensorRT-LLM, SGLang) that batches requests, manages the KV cache and selects kernels; where quantization formats become speed.

Intuition

Ask which wall you are against. Against the memory wall (few users, long answers), thin the weights and leave the activations alone. Against the compute wall (many users, long prompts), both operands of every multiply must be in a format the tensor core speaks, which is a shorter list. The grid on the slide is that list.

8.3 Slide 28: QAT, ternary weights, and quantized experts

When post-training rounding is not enough, train with the rounding in the loop.

What the slide shows The straight-through estimator: latent FP32 weights → quantized weights (4-bit, or $\{-1,0,1\}$) → forward pass and loss, with the backward arrow going straight from the loss to the latent weights as if the rounding were the identity. Text on quantization-aware training (LLM-QAT, Llama 3.2's

quantized releases) and BitNet b1.58. A panel on quantized MoE experts.

Details *Quantization-aware training.* Instead of rounding a finished model, train with the rounding inside the forward pass: the weights actually used are quant(w), so the loss already reflects the quantization, and the optimizer learns weights that round well. The difficulty is that rounding has zero gradient almost everywhere. The *straight-through estimator* (STE) ignores this: in the backward pass the gradient with respect to quant(w) is passed unchanged to w, as if the rounding were the identity. The latent weights w live and move in FP32; the forward pass only ever sees their rounded versions. The same device applies to activations. STE is a heuristic with no exact justification, and it works well enough that every QAT method uses it.

LLM-QAT (Liu et al. 2023) applied this to LLaMA models with a data-free twist: the training data for QAT is generated by the pretrained model itself, which avoids needing the original corpus, and the method quantizes weights, activations and the KV cache together down to 4 bits. Meta's Llama 3.2 1B and 3B quantized releases (October 2024) used a short QAT phase combined with LoRA adapters ("QLoRA" in Meta's naming, a different thing from Dettmers' QLoRA), alongside SpinQuant as the training-free alternative, and report 2–4× faster decode, 56% smaller models and 41% less memory than the BF16 originals.

BitNet b1.58 (Ma et al. 2024) takes QAT to its limit: every weight is in $\{-1,0,1\}$ (about 1.58 bits of information, $\log_2 3$) from the start of pretraining, with 8-bit activations, and the latent weights are kept in high precision and ternarized by absmean scaling in each forward pass. A matrix multiply with ternary weights needs no multiplications, only additions and subtractions, which is the point: the arithmetic becomes cheap enough for new hardware. The paper reports matching an FP16 LLaMA of the same size and training tokens from about 3B parameters on 100B tokens; BitNet b1.58 2B4T (2025) is a 2B-parameter model trained on 4T tokens in the same way. Whether ternary models hold up at frontier scale and data is an open question.

Quantized experts. In a mixture of experts almost all parameters are expert weights, which each token mostly does not use, and they quantize well. gpt-oss (OpenAI, 2025) post-trained its models with the expert weights in MXFP4 (4.25 bits), leaving attention, embeddings and the router in BF16; the 117B model fits on one 80 GB GPU and the 21B model runs in 16 GB. DeepSeek-V3 ships in FP8 from training (671 GB), and community 4-bit conversions bring it to roughly 350 GB with the usual loss on the hardest tasks. One component always stays in high precision: the router. Its few thousand numbers decide which experts fire, and rounding them would change the computation discontinuously, not just slightly.

Terms

straight-through estimator (STE) The backward-pass rule that treats a non-differentiable rounding as the identity, passing the gradient through to the latent weights.

latent weights The high-precision weights maintained by the optimizer in QAT, of which the forward pass uses only the rounded version.

LLM-QAT Data-free quantization-aware training for LLMs using self-generated data (Liu et al. 2023).

LoRA Low-rank adaptation: training small low-rank matrices added to frozen weights; Meta's quantized Llama 3.2 trained such adapters during QAT.

ternary weights Weights restricted to $\{-1,0,1\}$; $\log_2 3 \approx 1.58$ bits each. BitNet b1.58's format.

absmean scaling BitNet's ternarization: divide weights by their mean absolute value, then round to the nearest of $-1,0,1$.

gpt-oss OpenAI's 2025 open-weight MoE models (117B and 21B total) released with MXFP4 expert weights.

Intuition

Post-training quantization asks a finished model to live with a rounding it never saw; QAT lets the model grow up with it. The straight-through estimator is the fiction that makes this trainable: the forward pass is honest about the rounding, the backward pass pretends it is not there, and the latent weights learn to sit where rounding does least harm. BitNet is the fiction pushed to three values per weight.

8.4 Slide 29: Measuring the damage

Perplexity is the wrong instrument for a 4-bit model; the losses show up in long generations and hard tasks.

What the slide shows A sketch of score kept (as a percentage of the BF16 model) against bits per weight, for two kinds of evaluation: perplexity-based, and math, code and long-answer tasks. Both have a knee near 4 bits; the task-level curve falls earlier and faster below it. A checklist panel.

Details *Why perplexity under-reports.* Perplexity averages a per-token log-loss over ordinary text, most of which is easy and unaffected by small weight perturbations. Quantization errors matter where the model is near a decision boundary, and those cases are concentrated in hard tokens (the digit in a calculation, the identifier in a program) and compound over long generations: a model that is 0.5% worse per token is far more than 0.5% worse on a 2,000-token derivation that must be right at every step. Studies of quantized reasoning models (Liu et al. 2025, among others) find that 4-bit weight-only quantization of reasoning models costs measurable accuracy on AIME-style tasks while perplexity barely moves, with KV-cache quantization the most damaging component and the damage growing with the length of the generation.

The checklist.

- *Evaluate what you serve.* Long generations, sampled many times (variance is large), on math and code and on long-context tasks when the KV cache is quantized; with the same prompts, decoding parameters and system prompt as production.
- *Count the bits honestly.* Group scales and zero points are bits: INT4 with group-128 FP16 scales is 4.16 bits, MXFP4 4.25, NVFP4 4.5, NF4 with double quantization about 4.1. Compare at equal *total* bits, where a larger model at fewer bits usually wins (slide 25).
- *Expect the tax to grow with tokens per parameter.* A 15T-token 8B loses more at 4 bits than a 1T-token 8B of the same architecture; the over-trained small models everyone serves are the worst case.
- *Check the calibration set.* GPTQ, AWQ and SmoothQuant fit scales or corrections to a few hundred samples; a domain shift between calibration and use is a quality shift later. Calibrate on data like your traffic.
- *Report speed at your batch size.* Peak "with sparsity" TFLOP/s and batch-1 tokens per second describe different machines; the gain from weight-only quantization vanishes at large batch and the gain from activation quantization vanishes at small.

Terms

quantization tax The quality lost by quantizing, as a function of bits, model size and training data; the slide's vertical gap.

AIME The American Invitational Mathematics Examination, used as a benchmark of multi-step reasoning with exact answers; 30 questions per year, so results need many samples.

pass@1 (averaged) The accuracy of a single sampled answer, averaged over many samples per problem; the honest reasoning metric.

calibration shift Degradation caused by calibrating a quantization method on data unlike the deployment traffic.

equal-bits comparison Comparing models at the same parameters-times-bits budget rather than the same parameter count.

Intuition

Perplexity is a thermometer held at arm's length: it tells you the room is about the right temperature. The damage from rounding is a draught in one corner, where the hard problems sit. Measure there, with the window open for as long as a real user would keep it open.

9 Practice (slides 30–31)

9.1 Slide 30: Serving three models: the bill, with every lever applied

A dense 405B, a sparse 671B and a sparse-and-quantized 117B, as they ship.

What the slide shows A table of Llama 3.1 405B, DeepSeek-V3 and gpt-oss-120b: total and active parameters, the weights' shipped precision, weight bytes, the number of 80 GB GPUs needed for the weights alone, bytes read per token at batch 1, and KV cache per token. Three cards: what sparsity bought, what precision bought, what they still cost. A closing line on the 2025 pattern.

Details The table, all derived from the published configurations:

model	total / active	shipped weights	weight bytes	80 GB GPUs	bytes/token (batch 1)	KV/token
Llama 3.1 405B	405B / 405B	BF16 (FP8 variant)	810 GB (405)	11 (6)	810 GB (405)	504 KB
DeepSeek-V3	671B / 37B	FP8 from training	671 GB	9+	37 GB	69 KB
gpt-oss-120b	117B / 5.1B	MXFP4 experts, rest BF16	∼65 GB	1	∼3 GB	72 KB

GPU counts are for weights only; the KV cache, activations and serving overhead add to them, so real deployments use more. gpt-oss-120b's cache figure (36 layers, 8 KV heads, $d_h = 64$, BF16: $2 \times 36 \times 8 \times 64 \times 2 = 73{,}728$ bytes, 72 KB) is bounded in practice because half its layers use a 128-token banded attention whose cache does not grow with the context. Its bytes per token are approximate because the active 5.1B parameters are a mix of 4.25-bit expert weights and BF16 attention and embedding weights.

What sparsity bought. DeepSeek-V3 computes 37B parameters per token and reads 37 GB; a dense 405B reads 810 GB. gpt-oss-120b reads about 3 GB per token: a 117B-parameter model with the decode cost of a 5B dense model.

What precision bought. FP8 halved V3's footprint for free, because it trained in FP8. MXFP4 put 117B parameters on one 80 GB GPU; at BF16 the same model would be 234 GB on three.

What they still cost. Every expert must be resident, so V3 needs a multi-GPU node however few tokens it serves; its 69 KB per token of MLA cache is what makes 128k contexts affordable (9 GB per sequence rather than the 43 GB of a GQA 70B). The pattern of 2025, then: train sparse (MoE, MLA) in FP8, ship the experts at 4 bits, and attack the cache with sharing, FP8 and selection; the levers multiply on the bytes per token.

Terms

banded attention A sliding-window attention of fixed width (gpt-oss: 128 tokens) in alternate layers, whose cache does not grow with context.

weights as shipped The precision of the released checkpoint, which may be the training precision (V3), a post-training quantization (gpt-oss) or both variants (Llama 3.1).

Intuition

Three models, three answers to the same bill. The dense model pays in full. The MoE pays per token for a tenth of its weights and rents a whole node to keep the rest nearby. The sparse-and-quantized model pays per token for a twentieth, in quarter-size coin, and fits in one room. The bill is bytes per token, and the levers are the ways of making it smaller.

9.2 Slide 31: A practical guide

What to run, what to train in, and what to build, as of 2026.

What the slide shows Three columns of recommendations: inference (what to run), training (what to train in), architecture (what to build).

Details *Inference.* The default is FP8 W8A8 on Hopper or Blackwell with per-token dynamic scales: near-lossless and about twice BF16's throughput. When memory is the limit or the batch is small, use 4-bit weight-only quantization (AWQ, GPTQ, or k-quants for local inference) with a fused dequantizing kernel, and leave activations at 16 bits. On Blackwell, NVFP4 W4A4 once it has been validated on your own tasks. Always FP8 for the KV cache; GQA or MLA by architecture; selection or eviction past 128k tokens. And always evaluate the quantized model on long generations and hard tasks, not on perplexity (slide 29).

Training. The default is BF16 compute with FP32 master weights and optimizer. FP8 when the stack supports per-block scales (Transformer Engine's recipes, or DeepSeek-V3's: E4M3, tiles of 128, FP32 accumulation at intervals); expect 1.5–2$\times$ on the matmuls, less end to end, and keep the embeddings, norms, router and output head in higher precision. FP4 is a research setting in 2026: it needs stochastic rounding, Hadamard transforms, 2D block scales and sensitive layers kept higher, and it should be run with a BF16 control on the same tokens. Watch loss spikes and the gradient-norm history; they are the first symptom of a precision problem.

Architecture. MoE when you will serve at scale and have the memory: fine-grained experts, $k = 8$ of 128 or more, a shared expert, auxiliary-loss-free balancing, expert parallelism with the all-to-all overlapped. Dense when memory is the limit or the model is small. Small models by pruning and distilling a larger one rather than training from scratch. Long context with GQA or MLA plus a selection mechanism; fixed windows only for the cheap tier. And plan the bits early: a model trained in FP8 ships in FP8, and the question of how it will be quantized is best asked before pretraining, when the tokens-per-parameter ratio is being chosen.

Terms

fused kernel A kernel that performs dequantization and the matmul in one pass without writing intermediate results to memory.

gradient-norm history The sequence of global gradient norms over training; spikes and drifts in it precede loss divergence and are the usual first sign of numerical trouble.

BF16 control A reference run in BF16 on the same data and schedule, against which a low-precision run's loss curve is compared.

Intuition

The guide is three defaults and three exceptions. Run FP8 unless memory-bound, then 4-bit weights. Train BF16 unless the stack does blocks, then FP8. Build dense unless you serve at scale, then MoE. Everything else on the slide is the condition under which the exception applies.

10 Wrap-up (slides 32–33)

10.1 Slide 32: Summary: one cost model, two levers

Every method on these slides changes either how many numbers a token touches or how many bits each number has.

What the slide shows Six chevrons, one per technique family, each with what it buys and what it costs: MoE (capacity without FLOPs; memory, routing, all-to-all), pruning (smaller checkpoints, small models from big; no speedup unless structured), activation and attention sparsity (decode and long context linear in what is read; predictors, indexers, evaluations), formats (the right split of range and precision; outliers, accumulation width), low-bit training (2× per generation; scales, rounding, stability), quantization (4× fewer bytes at decode; task-level damage, honest bits). Then six points to remember.

Details The six points, with the slides they rest on.

1. Decode is memory-bound by two orders of magnitude; bytes per token, not FLOPs, is the number to cut, and both levers cut it (slide 2).
2. MoE trades memory for compute: quality follows total parameters at a discount, cost follows active ones; the optimal active fraction falls as models grow, and the open models went from a quarter active to a thirtieth in two years (slides 4–9).
3. Zeros are free only when the hardware can skip them: 2:4 and structured pruning run fast, unstructured pruning saves memory; the small models worth having are pruned and distilled from big ones (slides 10–13).
4. Outlier channels are the whole difficulty of low precision; per-block scales, smoothing and rotations are three ways to defuse them, and the same rotation lets FP4 training work (slides 19, 20, 24).
5. Precision has a scaling law: about 8 bits is compute-optimal for training, 4 bits is the knee for inference, and over-trained models pay the largest quantization tax (slides 25, 29).
6. The frontier open recipe composes all of it: sparse experts and compressed attention trained in FP8, shipped at 4–8 bits, with a cache that is shared, quantized and selectively read (slides 9, 17, 30).

Intuition

> If the lecture had one slide, it would be slide 2's roofline with the four points on it. Every technique since is a way to move a point: left and down the sloped part by reading less, or up the slope by reading smaller numbers, until the batch is large enough that the flat part takes over and only the tensor core's native formats matter.

10.2 Slide 33: Sources

Primary papers, checked against arXiv; † marks claims taken from vendor pages, model cards, blogs or commentary.

What the slide shows The reference list of the deck, grouped by section, with arXiv identifiers. Appendix C of these notes reproduces it in full with a line on why each paper is worth reading, in the order of the paper archive that accompanies the notes.

Details Three conventions used throughout the deck and notes. Curves labelled *illustrative* are sketches of a mechanism, not data; they have the right shape and no claim to the right numbers. Numbers labelled *derived* were computed from published configurations and the formulas in Appendix B, with KB $= 1024$

bytes for per-token cache sizes and GB $= 10^9$ bytes otherwise. Hardware peaks are dense unless stated and come from vendor datasheets; B200's per-GPU figures are derived from the eight-GPU HGX totals that NVIDIA publishes.

A note on what was *not* verifiable from primary sources at the time of writing, so that it can be re-checked: the OCP MX specification's publication date (the companion paper, Rouhani et al. 2023, was used instead); Hopper's accumulation precision (DeepSeek-V3's measurement, not an NVIDIA document); the 2026 model claims (Nemotron 3 Ultra, DeepSeek-V4, Kimi K3, Rubin), which rest on technical reports, model cards and announcements rather than peer-reviewed papers.

A Notation

symbol	meaning	notes
N_{total}	total parameters	everything stored; sets memory
N_{active}	active parameters	used by one token; sets FLOPs; equals N_{total} for dense models
N_{eff}	effective parameters	$N(1-e^{-P/\gamma})$, the precision-discounted count of Kumar et al.
D	training tokens	
P	precision in bits	P_w, P_a, P_{kv} for weights, activations, KV cache
E	number of experts	per MoE layer; also the irreducible loss in the Chinchilla form, by context
k	experts active per token	top-k routing
G	granularity	dense MLP width over expert width
g_i, f_i, P_i	gate, load fraction, mean gate	of expert i in a batch
b_i, γ	routing bias, its step	aux-loss-free balancing; γ also the precision constant, by context
CF, C	capacity factor, capacity	$C = \text{CF} \cdot T/E$ tokens per expert
L	sequence length	tokens in the context; also loss, by context
h, h_{kv}, d_h	query heads, KV heads, head dim	
W, X, H	weights, inputs, Hessian	layer-wise objective $\|WX - \hat{W}X\|^2$, $H = XX^\top$ (or $2XX^\top$)
S	sparsity	fraction removed; also the loss-scaling factor, by context
s, q	scale, code	$x \approx s \cdot q$
ε	machine epsilon	2^{-M} for M mantissa bits
E, M (format)	exponent, mantissa bits	as in E4M3
α	SmoothQuant migration strength; Switch aux-loss weight	by context
ρ	ridge point	peak FLOP/s over bandwidth, FLOP per byte

B Formulas at a glance

Cost model. training FLOPs $\approx 6N_{\text{active}}D$; decode FLOPs per token $\approx 2N_{\text{active}}$; weight bytes $= N_{\text{total}} \cdot \text{bits}/8$; KV bytes per token $= 2n_{\text{layers}}h_{kv}d_h \cdot \text{bits}/8$ (MLA: $n_{\text{layers}}(d_c + d_r) \cdot \text{bits}/8$).

Roofline. attainable FLOP/s $= \min(\text{bandwidth} \times \text{intensity},\ \text{peak})$; ridge $=$ peak/bandwidth (H100 BF16: $989/3.35 \approx 295$ FLOP/B). Batch-1 decode intensity $= 16/\text{bits}$ FLOP per byte of weights; batch B multiplies it by B.

Decode step time (memory-bound). $t \approx (\text{active weight bytes} + \text{KV bytes read})/\text{bandwidth}$.

MoE layer. $y = x + \sum_{i \in \text{TopK}(g)} g_i E_i(x)$, $g = \text{softmax}(W_r x)$; combinations $\binom{E}{k}$; capacity $C = \text{CF} \cdot T/E$.

Load balancing. $\mathscr{L}_{\text{aux}} = \alpha E \sum_i f_i P_i$ ($\alpha = 0.01$); z-loss $\frac{1}{T}\sum_t (\log \sum_j e^{x_{tj}})^2$; aux-loss-free: select by $g_i + b_i$, update $b_i \mp \gamma$ by load.

Pruning saliency. magnitude $|w|$; OBD $\frac{1}{2}H_{ii}w_i^2$; OBS $w_i^2/(2[H^{-1}]_{ii})$ with $\delta w = -\frac{w_i}{[H^{-1}]_{ii}} H^{-1} e_i$; Wanda $|W_{ij}| \, \|X_j\|_2$.

GPTQ column update. $W_{:,>q} \leftarrow W_{:,>q} - \frac{w_q - \text{quant}(w_q)}{[H^{-1}]_{qq}} [H^{-1}]_{q,>q}$.

Floating point. $x = (-1)^s 2^{e-\text{bias}}(1 + m/2^M)$; bias $= 2^{E-1} - 1$; $\varepsilon = 2^{-M}$; E4M3 max 448, E5M2 max 57,344, E2M1 values $\pm\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$.

Quantization. $q = \text{round}(x/s)$, $s = \max|x|/q_{\max}$; noise variance $s^2/12$; 6 dB per bit. Bits per number: MXFP4 $4 + 8/32 = 4.25$; NVFP4 $4 + 8/16 = 4.5$; INT4 g128 with FP16 scale and 4-bit zero ≈ 4.16; MXFP8 8.25.

SmoothQuant. $s_j = \max|X_{:,j}|^{\alpha} / \max|W_{j,:}|^{1-\alpha}$, $\alpha = 0.5$; $XW = (X\,\text{diag}(s)^{-1})(\text{diag}(s)W)$.

Rotation. $XW = (XH)(H^{\top}W)$ for orthogonal H; Hadamard entries $\pm 1/\sqrt{d}$.

Mixed precision. bytes per parameter $= 2 + 2 + 4 + 4 + 4 = 16$ (BF16 weights, BF16 grads, FP32 master, m, v); 12 with BF16 moments.

Precision law (Kumar et al.). $N_{\text{eff}} = N(1 - e^{-P/\gamma})$; $L = A/N_{\text{eff}}^{\alpha} + B/D^{\beta} + E$; $\delta_{\text{PTQ}} \propto D^{\gamma_D} N^{-\gamma_N} e^{-P_{\text{post}}/\gamma_{\text{post}}}$; compute-optimal $P \approx 7$–8 bits (integer fits).

Sparsity law (Frantar et al.). $L(S,N,D) = \left(a_S(1-S)^{b_S} + c_S\right)\left(\frac{1}{N}\right)^{b_N} + \left(\frac{a_D}{D}\right)^{b_D} + c$; optimal S rises with D/N.

Worked numbers (derived). Llama 3 70B KV: $2 \cdot 80 \cdot 8 \cdot 128 \cdot 2 = 327{,}680$ B $= 320$ KB/token, 42.9 GB at 128k. DeepSeek-V3 KV: $(512 + 64) \cdot 61 \cdot 2 = 70{,}272$ B ≈ 69 KB/token. V3 training: $6 \cdot 37\text{B} \cdot 14.8\text{T} \approx 3.3 \times 10^{24}$ FLOPs.

C Exercises

1. **Roofline.** An H100 SXM has 989 TFLOP/s dense BF16 and 3.35 TB/s. For a 70B dense model, compute the batch-1 decode time per token at BF16, FP8 and 4-bit weights, ignoring the KV cache. At what batch size does the BF16 model reach the ridge? Repeat for a B200 (2,250 TFLOP/s, 8 TB/s) and comment on how the ridge moved.

2. **MoE accounting.** DeepSeek-V3 has 61 layers, 256 routed and 1 shared expert per MoE layer, $k = 8$. Using 671B total and 37B active, estimate the parameters per expert (assume attention and embeddings are a small fraction) and the batch size at which, in expectation, every expert is touched at least once per step. What does that imply for the bytes per token at that batch?

3. **Combinations.** Show that at equal active width, splitting E experts into mE experts of width $1/m$ and activating mk raises the number of combinations from $\binom{E}{k}$ to $\binom{mE}{mk}$. Evaluate for $E = 8$, $k = 2$, $m = 4$ and $m = 8$.

4. **Balance.** Prove that $\sum_i f_i P_i$ with $\sum f_i = \sum P_i = 1$ is minimized when $f = P = (1/E, \ldots, 1/E)$ under the constraint that f and P are monotonically related (more probability means more load), and compute the minimum. Why is the loss written as a product of a non-differentiable and a differentiable term?

5. **OBS.** Starting from $\delta\mathscr{L} = \frac{1}{2}\delta w^\top H \delta w$ and the constraint $e_i^\top \delta w + w_i = 0$ (weight i is set to zero), derive the OBS update and saliency by Lagrange multipliers.

6. **2:4.** A weight matrix is stored in 2:4 compressed form. Count the bits per original weight at FP16 values plus a 2-bit index per kept value, and compare with the dense FP16 storage. What is the storage saving, and what is the compute saving?

7. **KV cache.** Compute the KV bytes per token for Llama 3 8B (32 layers, 8 KV heads, $d_h = 128$) at BF16, FP8 and 2-bit (KIVI-style, ignore scales), and the total at 128k tokens. How many concurrent 128k sequences fit in 80 GB alongside the FP8 weights (8B parameters)?

8. **Selection arithmetic.** With DSA selecting $k = 2048$ tokens, what fraction of a 128k cache is read per query? If the indexer costs the equivalent of reading 4% of the cache, at what context length does selection stop paying?

9. **Formats.** List all positive values of E2M1 and verify that the maximum relative rounding error is 25%. Do the same for a 4-bit integer scaled to the same maximum (6), and find the value below which the integer's absolute error exceeds the float's relative error.

10. **Block scales.** A block of 32 weights contains one outlier of magnitude 8 and thirty-one values of magnitude about 0.1. With one E8M0 scale for the block and E2M1 codes, what codes do the small values receive? Repeat with blocks of 16 and with the outlier removed by a Hadamard rotation that spreads its energy evenly over the block (magnitude $8/\sqrt{32}$ added to each entry).

11. **Loss scaling.** Gradients of a layer are distributed roughly log-uniformly between 10^{-9} and 10^{-1}. What fraction underflows in FP16 (smallest normal 6.1×10^{-5}) without loss scaling, and with a scale of $S = 2^{12}$? Why is the question moot in BF16?

12. **Accumulation.** A dot product of length 7,168 (DeepSeek-V3's hidden size) sums FP8 products of magnitude about 1. If the accumulator keeps 14 bits of mantissa, after how many terms does adding one

more product of magnitude 1 to the running sum lose a full bit of it? Explain why promoting to FP32 every 128 terms fixes this.

13. **Precision law.** With $N_{\text{eff}} = N(1 - e^{-P/\gamma})$, $\gamma = 2.7$, compute the effective-parameter discount at $P = 4, 6, 8, 16$. At a fixed budget proportional to $N \cdot P$, compare N at 16 bits with $4N$ at 4 bits and $2N$ at 8 bits in effective parameters, and say which wins.

14. **Quantization tax.** Two 8B models of identical architecture are trained on 1T and 15T tokens. Using the proportionality $\delta_{\text{PTQ}} \propto D^{\gamma_D}$ with $\gamma_D \approx 0.5$ (an order-of-magnitude assumption), by what factor is the loss increase from 4-bit PTQ larger for the second? What evaluation would you run to check the prediction, and why is perplexity a poor choice?

15. **Honest bits.** Compute the bits per weight of: INT4 with a group of 32 and an FP16 scale; INT4 with a group of 128, an FP16 scale and a 4-bit zero point; NF4 with a group of 64 and an 8-bit scale per block plus one FP32 scale per 256 blocks (QLoRA's double quantization); MXFP4; NVFP4. Rank them, then rank them by the resolution of their scales.

16. **Serving bill.** Repeat slide 30's table for a hypothetical 1T-total, 50B-active MoE shipped in NVFP4 experts (4.5 bits) with BF16 attention amounting to 5B parameters, GQA with 8 KV heads of dimension 128 over 60 layers at FP8 cache. Give weight bytes, 80 GB GPUs for weights, bytes per token at batch 1, and KV bytes per token.

D Reading list by section

The papers behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier are included in the paper archive, the zip file that accompanies these notes; entries without one are listed with a URL. Items marked *secondary source* are vendor pages, model cards or blog posts.

1. Overview and the cost model (slides 1-3)

- **Williams, Waterman and Patterson (2009).** *Roofline: an insightful visual performance model for multicore architectures*. `https://doi.org/10.1145/1498765.1498785`. The model behind slide 2: attainable throughput against arithmetic intensity. Communications of the ACM 52(4).
- **Kaplan et al. (2020).** *Scaling laws for neural language models*. arXiv:2001.08361. Source of the 6ND compute count and the loss-versus-resource power laws the precision law extends.
- **Hoffmann et al. (2022).** *Training compute-optimal large language models*. arXiv:2203.15556. The parametric loss form L = A/Nˆa + B/Dˆb + E into which effective parameters are substituted on slide 25.
- **Pope et al. (2022).** *Efficiently scaling transformer inference*. arXiv:2211.05102. The clearest published roofline analysis of prefill and decode, batching, and the KV cache; the quantitative background of slides 2 and 15.
- **Kwon et al. (2023).** *Efficient memory management for large language model serving with PagedAttention*. arXiv:2309.06180. vLLM: how a serving stack manages the KV cache; the system in which most quantization formats on slide 27 are deployed.

2. Mixture of experts (slides 4-9)

- **Jacobs, Jordan, Nowlan and Hinton (1991).** *Adaptive mixtures of local experts*. `https://doi.org/10.1162/neco.1991.3.1.79`. The origin of the idea: a gating network that assigns inputs to expert networks. Neural Computation 3(1).
- **Shazeer et al. (2017).** *Outrageously large neural networks: the sparsely-gated mixture-of-experts layer*. arXiv:1701.06538. Sparse top-k gating at scale (in an LSTM LM); introduces the load-balancing losses every later paper inherits.
- **Lepikhin et al. (2020).** *GShard: scaling giant models with conditional computation and automatic sharding*. arXiv:2006.16668. Top-2 MoE in transformers at 600B parameters; expert parallelism and capacity factors as used since.
- **Fedus, Zoph and Shazeer (2021).** *Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity*. arXiv:2101.03961. Top-1 routing, the auxiliary loss with alpha = 0.01, and the 7x pretraining speedup; the reference MoE recipe of 2021.
- **Du et al. (2021).** *GLaM: efficient scaling of language models with mixture-of-experts*. arXiv:2112.06905. 1.2T parameters, 64 experts, top-2, trained with a third of GPT-3's energy; early evidence that MoE quality tracks total parameters.
- **Zoph et al. (2022).** *ST-MoE: designing stable and transferable sparse expert models*. arXiv:2202.08906. The router z-loss and a careful study of MoE training stability and fine-tuning.
- **Zhou et al. (2022).** *Mixture-of-experts with expert choice routing*. arXiv:2202.09368. Experts pick tokens instead of tokens picking experts: perfect balance by construction.

- **Clark et al. (2022).** *Unified scaling laws for routed language models*. arXiv:2202.01169. The first MoE scaling law; recommends 64-128 experts at the scales studied and quantifies diminishing returns.
- **Fedus, Dean and Zoph (2022).** *A review of sparse expert models in deep learning*. arXiv:2209.01667. A survey of the MoE literature up to 2022, useful for the history and the terminology.
- **Gale et al. (2022).** *MegaBlocks: efficient sparse training with mixture-of-experts*. arXiv:2211.15841. Dropless MoE: block-sparse kernels that let experts take variable-size batches, no token dropping.
- **Jiang et al. (2024).** *Mixtral of experts*. arXiv:2401.04088. The open 8-expert top-2 model (47B total, 13B active) that made MoE mainstream in open weights.
- **Dai et al. (2024).** *DeepSeekMoE: towards ultimate expert specialization in mixture-of-experts language models*. arXiv:2401.06066. Fine-grained expert segmentation and shared experts: the design of every DeepSeek MoE and most 2025 open models.
- **Krajewski et al. (2024).** *Scaling laws for fine-grained mixture of experts*. arXiv:2402.07871. Granularity as a scaling-law variable; MoE beats dense at every scale with granularity tuned.
- **DeepSeek-AI (2024).** *DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model*. arXiv:2405.04434. Introduces multi-head latent attention (93.3% KV-cache reduction) with a 236B/21B MoE.
- **Wang et al. (2024).** *Auxiliary-loss-free load balancing strategy for mixture-of-experts*. arXiv:2408.15664. The per-expert bias update used by DeepSeek-V3 to balance load without a loss term.
- **DeepSeek-AI (2024).** *DeepSeek-V3 technical report*. arXiv:2412.19437. 671B/37B MoE, MLA, aux-loss-free balancing, DualPipe, and the FP8 training recipe of slide 23; the central reference of the deck.
- **Abnar et al. (2025).** *Parameters vs FLOPs: scaling laws for optimal sparsity for mixture-of-experts language models*. arXiv:2501.12370. Total parameters and FLOPs as separate axes; optimal sparsity rises with model size at fixed compute.
- **Ludziejewski et al. (2025).** *Joint MoE scaling laws: mixture of experts can be memory efficient*. arXiv:2502.05172. Adds memory to the objective; MoE can be the memory-efficient choice too.
- **Qwen Team (2025).** *Qwen3 technical report*. arXiv:2505.09388. 235B/22B MoE with 128 experts and no shared expert; thinking budgets; strong-to-weak distillation.
- **Kimi Team (2025).** *Kimi K2: open agentic intelligence*. arXiv:2507.20534. 1.04T/32B MoE with 384 experts, MLA and the MuonClip optimizer.
- **OpenAI (2025).** *gpt-oss-120b and gpt-oss-20b model card*. arXiv:2508.10925. 128-expert top-4 MoE shipped with MXFP4 expert weights; 117B on one 80 GB GPU.
- **DeepSeek-AI (2026).** *DeepSeek-V4: towards highly efficient million-token context intelligence*. arXiv:2606.19348. 1.6T/49B and 284B/13B MoEs with compressed sparse attention and FP4 expert weights (2026).
- **Meta AI (2025).** *The Llama 4 herd (blog post)*. `https://ai.meta.com/blog/llama-4-multimodal-intelligence/`. Llama 4 Maverick (400B/17B, 128 experts + shared, top-1) and Scout; alternating dense and MoE layers. Secondary source.
- **Qwen Team (2025).** *Qwen3-Next-80B-A3B model card*. `https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct`. 512 experts, 10 routed + 1 shared, 80B/3B, hybrid Gated DeltaNet attention. Secondary source.

3. Weight pruning (slides 10-13)

- **LeCun, Denker and Solla (1989).** *Optimal brain damage*. https://proceedings.neurips.cc/paper_files/paper/1989/file/6c9882bbac1c7093bd25041881277658-Paper.pdf. Second-order (diagonal Hessian) saliency for pruning. NeurIPS 1989.
- **Hassibi and Stork (1992).** *Second order derivatives for network pruning: Optimal Brain Surgeon*. https://proceedings.neurips.cc/paper_files/paper/1992/file/303ed4c69846ab36c2904d3ba8573050-Paper.pdf. Full inverse-Hessian saliency with the compensating update; the ancestor of SparseGPT and GPTQ. NeurIPS 1992.
- **Han, Pool, Tran and Dally (2015).** *Learning both weights and connections for efficient neural networks*. arXiv:1506.02626. Iterative magnitude pruning with retraining: 9x on AlexNet, 13x on VGG-16, no accuracy loss.
- **Han, Mao and Dally (2016).** *Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding*. arXiv:1510.00149. Pruning plus quantization plus coding, 35-49x smaller; the first paper to compose the two levers.
- **Frankle and Carbin (2019).** *The lottery ticket hypothesis: finding sparse, trainable neural networks*. arXiv:1803.03635. Sparse subnetworks at 10-20% of the size train to full accuracy from their original initialization.
- **Hoefler et al. (2021).** *Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks*. arXiv:2102.00554. A comprehensive survey of sparsity methods and hardware; the reference map of the field before LLMs.
- **Mishra et al. (2021).** *Accelerating sparse deep neural networks*. arXiv:2104.08378. The 2:4 pattern, its compressed format, and the train-prune-retrain recipe for Ampere sparse tensor cores.
- **Frantar and Alistarh (2023).** *SparseGPT: massive language models can be accurately pruned in one-shot*. arXiv:2301.00774. One-shot OBS-style layer-wise pruning of 175B models to 50-60% in hours on one GPU.
- **Sun et al. (2023).** *A simple and effective pruning approach for large language models*. arXiv:2306.11695. Wanda: prune by |weight| x activation norm with no weight updates; nearly matches SparseGPT at 50%.
- **Frantar et al. (2023).** *Scaling laws for sparsely-connected foundation models*. arXiv:2309.08520. A joint law in sparsity, parameters and data; optimal sparsity rises with training data per parameter.
- **Xia et al. (2023).** *Sheared LLaMA: accelerating language model pre-training via structured pruning*. arXiv:2310.06694. LLaMA-2 7B to 2.7B/1.3B with learned masks and 50B tokens of continued pretraining, 3% of from-scratch compute.
- **Ma, Fang and Wang (2023).** *LLM-Pruner: on the structural pruning of large language models*. arXiv:2305.11627. Gradient-based structured pruning of LLMs with LoRA recovery; an early LLM structured-pruning baseline.
- **Ashkboos et al. (2024).** *SliceGPT: compress large language models by deleting rows and columns*. arXiv:2401.15024. Structured pruning by orthogonal transformation then slicing; introduces the computational-invariance idea reused in QuaRot.
- **Muralidharan et al. (2024).** *Compact language models via pruning and knowledge distillation*. arXiv:2407.14679. Minitron: Nemotron-4 15B to 8B/4B with up to 40x fewer tokens; width beats depth; distillation beats fine-tuning.

- **Sreenivas et al. (2024).** *LLM pruning and distillation in practice: the Minitron approach*. arXiv:2408.11796. The recipe applied to Llama 3.1 8B and Mistral NeMo 12B, with teacher correction.

4. Activation and attention sparsity (slides 14-17)

- **Li et al. (2022).** *The lazy neuron phenomenon: on emergence of activation sparsity in transformers*. arXiv:2210.06313. Measures the emergent sparsity of ReLU MLPs (3% nonzero in T5-Base) and shows it grows with scale.
- **Mirzadeh et al. (2023).** *ReLU strikes back: exploiting activation sparsity in large language models*. arXiv:2310.04564. ReLUfication of LLMs: up to 3x less inference computation with little quality loss.
- **Liu et al. (2023).** *Deja Vu: contextual sparsity for efficient LLMs at inference time*. arXiv:2310.17157. Predict the live neurons and heads per input; 2x over FasterTransformer on OPT-175B.
- **Song et al. (2023).** *PowerInfer: fast large language model serving with a consumer-grade GPU*. arXiv:2312.12456. Hot/cold neuron partitioning between GPU and CPU using activation sparsity; the systems follow-up to Deja Vu.
- **Lee et al. (2024).** *CATS: contextually-aware thresholding for sparsity in large language models*. arXiv:2404.08763. Threshold the gated MLP for 50% sparsity within 1-2% of the base model, training-free.
- **Liu et al. (2024).** *Training-free activation sparsity in large language models*. arXiv:2408.14690. TEAL: magnitude thresholds model-wide, 40-50% sparsity, 1.5-1.8x decode speedup.
- **Child et al. (2019).** *Generating long sequences with sparse transformers*. arXiv:1904.10509. Strided and fixed sparse attention patterns, O(n sqrt n); the first sparse-attention transformer.
- **Beltagy, Peters and Cohan (2020).** *Longformer: the long-document transformer*. arXiv:2004.05150. Sliding window plus global tokens, linear in length.
- **Zaheer et al. (2020).** *Big Bird: transformers for longer sequences*. arXiv:2007.14062. Window + global + random blocks; shows the sparse pattern is a universal approximator.
- **Shazeer (2019).** *Fast transformer decoding: one write-head is all you need*. arXiv:1911.02150. Multi-query attention: share keys and values across heads to shrink the cache.
- **Ainslie et al. (2023).** *GQA: training generalized multi-query transformer models from multi-head checkpoints*. arXiv:2305.13245. Grouped-query attention, the compromise used by Llama 3 and most 2024 models.
- **Dao et al. (2022).** *FlashAttention: fast and memory-efficient exact attention with IO-awareness*. arXiv:2205.14135. Exact attention tiled to avoid materializing the L x L matrix; the kernel every sparse-attention speedup is measured against.
- **Jiang et al. (2023).** *Mistral 7B*. arXiv:2310.06825. Sliding-window attention (4,096 tokens) with a rolling cache, plus GQA, in a widely used open model.
- **Zhang et al. (2023).** *H2O: heavy-hitter oracle for efficient generative inference of large language models*. arXiv:2306.14048. KV eviction keeping the tokens with the most accumulated attention; 20% budgets retain accuracy.
- **Xiao et al. (2023).** *Efficient streaming language models with attention sinks*. arXiv:2309.17453. Attention sinks: keep the first four tokens plus a window and stream millions of tokens.

- **Li et al. (2024).** *SnapKV: LLM knows what you are looking for before generation*. arXiv:2404.14469. Compress the prompt's KV cache per head from the attention of the last prompt tokens.
- **Tang et al. (2024).** *Quest: query-aware sparsity for efficient long-context LLM inference*. arXiv:2406.10774. Per-query page selection by score bounds; selection without a learned indexer.
- **Liu et al. (2024).** *KIVI: a tuning-free asymmetric 2bit quantization for KV cache*. arXiv:2402.02750. Keys per channel, values per token: 2-bit KV cache with small loss.
- **Yuan et al. (2025).** *Native sparse attention: hardware-aligned and natively trainable sparse attention*. arXiv:2502.11089. Compressed, selected and sliding-window branches, trained from scratch; 9x forward and 6x backward kernel speedups at 64k.
- **Lu et al. (2025).** *MoBA: mixture of block attention for long-context LLMs*. arXiv:2502.13189. Route each query to its top-k blocks of the cache, as a router routes tokens to experts.
- **DeepSeek-AI (2025).** *DeepSeek-V3.2: pushing the frontier of open large language models*. arXiv:2512.02556. DeepSeek Sparse Attention: a lightning indexer selecting the top 2048 tokens per query on top of MLA.
- **Hsieh et al. (2024).** *RULER: what's the real context size of your long-context language models?*. arXiv:2404.06654. Thirteen long-context tasks in four categories; most models fall short of their claimed lengths.

5. Number formats and outliers (slides 18-21)

- **Micikevicius et al. (2022).** *FP8 formats for deep learning*. arXiv:2209.05433. Defines E4M3 and E5M2, their special values and the rationale; the format reference for slide 18.
- **Rouhani et al. (2023).** *Microscaling data formats for deep learning*. arXiv:2310.10537. The MX block formats (32-element blocks, E8M0 scale) with training and inference results; companion to the OCP spec.
- **Open Compute Project (2023).** *OCP Microscaling Formats (MX) specification v1.0*. `https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf`. The normative definition of MXFP8/6/4 and MXINT8. Not on arXiv.
- **NVIDIA (2025).** *Introducing NVFP4 for efficient and accurate low-precision inference (developer blog)*. `https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/`. E2M1 elements, 16-element blocks with E4M3 scales, FP32 per-tensor scale. Secondary source.
- **Dettmers et al. (2022).** *LLM.int8(): 8-bit matrix multiplication for transformers at scale*. arXiv:2208.07339. Discovers the outlier features that emerge around 6.7B parameters and the mixed-precision decomposition that handles them.
- **Bondarenko, Nagel and Blankevoort (2021).** *Understanding and overcoming the challenges of efficient transformer quantization*. arXiv:2109.12948. Early analysis of why transformer activations (and their outliers) resist INT8, with per-embedding-group fixes.
- **Xiao et al. (2022).** *SmoothQuant: accurate and efficient post-training quantization for large language models*. arXiv:2211.10438. Migrate activation difficulty into the weights with per-channel scales; W8A8 up to 1.56x faster.
- **Sun et al. (2024).** *Massive activations in large language models*. arXiv:2402.17762. A handful of activations at specific tokens act as fixed biases and attention sinks; the mechanism behind the outliers.

- **Ashkboos et al. (2024).** *QuaRot: outlier-free 4-bit inference in rotated LLMs*. arXiv:2404.00456. Hadamard rotations folded into the weights; 4-bit weights, activations and KV cache with at most 0.47 perplexity loss on Llama-2 70B.
- **Liu et al. (2024).** *SpinQuant: LLM quantization with learned rotations*. arXiv:2405.16406. Learned rotations on the Stiefel manifold; used in Meta's Llama 3.2 quantized releases.
- **Dettmers et al. (2023).** *QLoRA: efficient finetuning of quantized LLMs*. arXiv:2305.14314. NF4, double quantization and paged optimizers; fine-tune a 65B model on one 48 GB GPU.

6. Low-precision training (slides 22-24)

- **Micikevicius et al. (2018).** *Mixed precision training*. arXiv:1710.03740. FP32 master weights and loss scaling: the recipe that made FP16 training work and that every later format inherits.
- **Kalamkar et al. (2019).** *A study of BFLOAT16 for deep learning training*. arXiv:1905.12322. Why BF16's FP32 range removes the need for loss scaling at the cost of mantissa bits.
- **Rajbhandari et al. (2020).** *ZeRO: memory optimizations toward training trillion parameter models*. arXiv:1910.02054. The 16-bytes-per-parameter accounting of mixed-precision training and how to partition it.
- **Peng et al. (2023).** *FP8-LM: training FP8 large language models*. arXiv:2310.18313. FP8 weights, gradients, optimizer states and communication in GPT-175B training; an early full-stack FP8 recipe.
- **Fishman et al. (2024).** *Scaling FP8 training to trillion-token LLMs*. arXiv:2409.12517. Instabilities that appear only after hundreds of billions of tokens in FP8 (SwiGLU outlier amplification) and their fix.
- **Xi et al. (2023).** *Training transformers with 4-bit integers*. arXiv:2306.11987. INT4 forward and backward with Hadamard quantizers and leverage-score sampling; a precursor of the FP4 recipes.
- **Tseng et al. (2025).** *Training LLMs with MXFP4*. arXiv:2502.20586. Stochastic rounding plus random Hadamard transforms for near-lossless MXFP4 training; more than half the FLOPs in MXFP4.
- **NVIDIA (2025).** *Recipes for pre-training LLMs with MXFP8*. arXiv:2506.08027. Round the block scale up, not down; an 8B model on 15T tokens matches BF16.
- **NVIDIA (2025).** *Pretraining large language models with NVFP4*. arXiv:2509.25149. 12B on 10T tokens in NVFP4: 2D block scales, Hadamard transforms, stochastic rounding, 16% of layers kept higher.
- **Castro et al. (2025).** *Quartet: native FP4 training can be optimal for large language models*. arXiv:2505.14669. End-to-end FP4 training with Blackwell kernels, framed with an accuracy-versus-efficiency law.
- **NVIDIA (2026).** *Nemotron 3 Ultra technical report*. `https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf`. 550B/55B model pretrained in NVFP4 on 20T tokens with a loss gap under 0.4% (2026). Secondary source; not on arXiv.

7. Scaling laws for precision (slide 25)

- **Kumar et al. (2024).** *Scaling laws for precision*. arXiv:2411.04330. Effective parameters N(1 - eˆ-P/gamma), the 7-8 bit compute-optimal precision, and PTQ degradation growing with data.

- **Dettmers and Zettlemoyer (2023).** *The case for 4-bit precision: k-bit inference scaling laws*. arXiv:2212.09720. 35,000 experiments from 19M to 176B parameters: 4-bit weights are almost universally optimal per total model bit.
- **Ouyang et al. (2024).** *Low-bit quantization favors undertrained LLMs: scaling laws for quantized LLMs with 100T training tokens*. arXiv:2411.17691. Independent confirmation that quantization damage grows with tokens per parameter.
- **Sun et al. (2025).** *Scaling laws for floating-point quantization training*. arXiv:2501.02423. Exponent and mantissa bits as separate variables; cost-optimal precision between 4 and 8 bits.

8. Quantization for inference (slides 26-29)

- **Gholami et al. (2021).** *A survey of quantization methods for efficient neural network inference*. arXiv:2103.13630. The standard survey of quantization concepts (uniform/non-uniform, symmetric/asymmetric, PTQ/QAT) before the LLM era.
- **Frantar and Alistarh (2022).** *Optimal brain compression: a framework for accurate post-training quantization and pruning*. arXiv:2208.11580. OBQ/OBC: the per-weight quantize-and-compensate method that GPTQ and SparseGPT scale up.
- **Frantar et al. (2022).** *GPTQ: accurate post-training quantization for generative pre-trained transformers*. arXiv:2210.17323. OPT-175B to 3-4 bits in four GPU-hours; column-wise error feedback with a shared inverse Hessian.
- **Lin et al. (2023).** *AWQ: activation-aware weight quantization for LLM compression and acceleration*. arXiv:2306.00978. Protect the 1% salient channels by scaling before rounding; 4-bit weights, no backprop, and the TinyChat kernels.
- **Kim et al. (2023).** *SqueezeLLM: dense-and-sparse quantization*. arXiv:2306.07629. Sensitivity-weighted non-uniform quantization with outliers kept sparse; another route to 3-4 bits.
- **Chee et al. (2023).** *QuIP: 2-bit quantization of large language models with guarantees*. arXiv:2307.13304. Incoherence processing by random orthogonal matrices with a theoretical bound; the start of the rotation line.
- **Tseng et al. (2024).** *QuIP#: even better LLM quantization with Hadamard incoherence and lattice codebooks*. arXiv:2402.04396. Hadamard incoherence plus E8 lattice codebooks for 2-4 bit weights.
- **Egiazarian et al. (2024).** *Extreme compression of large language models via additive quantization*. arXiv:2401.06118. AQLM: additive codebooks; the first Pareto-optimal scheme below 3 bits per weight.
- **Frantar et al. (2024).** *MARLIN: mixed-precision auto-regressive parallel inference on large language models*. arXiv:2408.11743. The W4A16 kernel that reaches near-ideal 4x speedup up to batch 16-32.
- **Zhao et al. (2023).** *Atom: low-bit quantization for efficient and accurate LLM serving*. arXiv:2310.19102. W4A4 serving with outlier channels in INT8; an early activation-quantized serving design.
- **Lin et al. (2024).** *QServe: W4A8KV4 quantization and system co-design for efficient LLM serving*. arXiv:2405.04532. 4-bit weights, 8-bit activations, 4-bit KV cache with register-level dequantization; 1.2-3.5x over TensorRT-LLM.
- **Liu et al. (2023).** *LLM-QAT: data-free quantization aware training for large language models*. arXiv:2305.17888. QAT for LLMs using self-generated data; weights, activations and KV cache to 4 bits.

- **Wang et al. (2023).** *BitNet: scaling 1-bit transformers for large language models*. arXiv:2310.11453. Binary weights from scratch with the straight-through estimator; the precursor of b1.58.
- **Ma et al. (2024).** *The era of 1-bit LLMs: all large language models are in 1.58 bits*. arXiv:2402.17764. Ternary weights -1,0,1 from pretraining; matches FP16 LLaMA from 3B parameters at the same tokens.
- **Ma et al. (2025).** *BitNet b1.58 2B4T technical report*. arXiv:2504.12285. A 2B-parameter ternary model trained on 4T tokens, with CPU and GPU kernels.
- **Liu et al. (2025).** *Quantization hurts reasoning? An empirical study on quantized reasoning models*. arXiv:2504.04823. Measures the damage of weight, activation and KV quantization on reasoning models over long generations; evidence for slide 29.
- **Meta AI (2024).** *Introducing quantized Llama models with increased speed and a reduced memory footprint (blog post)*. `https://ai.meta.com/blog/meta-llama-quantized-lightweight-models/`. Llama 3.2 1B/3B with QAT+LoRA and SpinQuant: 2-4x faster, 56% smaller, 41% less memory. Secondary source.

9. Practice: models and systems (slides 30-31)

- **Grattafiori et al. (2024).** *The Llama 3 herd of models*. arXiv:2407.21783. The dense 405B reference point of slides 9 and 30: GQA, 126 layers, BF16 and FP8 variants.
- **NVIDIA (2022).** *Transformer Engine documentation: FP8 primer and recipes*. `https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html`. Delayed and current scaling, the hybrid E4M3/E5M2 recipe, amax history. Secondary source.
- **NVIDIA and AMD (2020).** *GPU datasheets: A100, H100, HGX B200, MI300X, MI355X*. `https://www.nvidia.com/en-us/data-center/`. The peak-throughput and bandwidth figures of slide 21; dense values are half the 'with sparsity' headlines. Secondary source.
- **Moonshot AI (2026).** *Kimi K3 model card*. `https://huggingface.co/moonshotai/Kimi-K3`. 2.8T/104B with 16 of 896 experts plus 2 shared; MXFP4/MXFP8 QAT from SFT onward (2026). Secondary source.

Index of terms

"with sparsity" peak, 18
2:4 sparsity, 18
2D block scaling, 38

absmax scaling, 30
absmean scaling, 44
accumulation precision, 33
activation sparsity, 23
active fraction, 11
active parameters, 5
Adam moments, 35
AIME, 45
all-reduce, 36
all-to-all, 12
arithmetic intensity, 5
Atom, 43
attention sink, 24
auxiliary loss, 10
AWQ, 42

banded attention, 47
batch, 5
BF16, 29
BF16 control, 48
binade, 28
block, 26
block floating point, 30
block-sparse attention, 24

calibration set, 19
calibration shift, 45
capacity factor (CF), 10
Chinchilla loss form, 40
Cholesky factorization, 19
codebook, 42
compressed sparse format, 18
compute-bound, 5
compute-optimal precision, 40
compute-optimal sparsity, 13
contextual sparsity, 23
continued pretraining, 21
current scaling, 37

decode, 5
DeepSeek Sparse Attention (DSA), 26
delayed scaling, 37
dense peak, 33
depth pruning, 20
dequantization, 42
distillation, 20
double quantization, 30
dropless MoE, 10
DualPipe, 12
dynamic quantization, 43

E2M1, 29
E4M3, E5M2, 29
E8M0, 30
effective parameters, 40
equal-bits comparison, 46
eviction, 25
expert, 6
expert choice, 10
expert parallelism (EP), 12
exponent, 28

feed-forward block (MLP), 8
fine-grained experts, 11
FP8, 14
fused kernel, 48

gate, 8
GELU, SiLU, SwiGLU, 23
GEMM, 37
gpt-oss, 44
GPTQ, 42
GPU-hour, 14
gradient-norm history, 48
granularity (G), 11
group quantization, 30
grouped-query attention (GQA), 24

H800, 14
Hadamard matrix, 32
HBM, 5
HBM3e, HBM4, 33
heavy hitter, 25
Hessian, 17

importance score, 20
incoherence, 32
indexer, 25

INT8, INT4, 29
iterative pruning, 16

k-quants, 42
KIVI, 27
KL divergence, 20
KV cache, 5
KV quantization, 27

latent weights, 44
layer-wise reconstruction, 19
lazy neuron, 23
LLM-QAT, 44
load balancing, 10
LoRA, 44
loss scaling, 35
lottery ticket hypothesis, 17

machine epsilon, 29
magnitude pruning, 16
mantissa (significand), 28
Marlin, 42
massive activations, 32
master weights, 35
memory-bound, 5
migration strength (alpha), 32
mixed-precision decomposition, 32
mixed-precision training, 35
mixture of experts (MoE), 4
MoBA, 26
MoE layer, 11
MoE scaling law, 13
multi-head attention (MHA), 24
multi-head latent attention (MLA), 14, 24
multi-query attention (MQA), 24
MX (Microscaling), 30
MX, NVFP4, 7
MXFP4, 38
MXFP8, 37

native format, 33
native sparse attention (NSA), 26
NF4, 30
node-limited routing, 12
NVFP4, 30

OBQ, 42
one-shot pruning, 17
Optimal Brain Damage (OBD), 17
Optimal Brain Surgeon (OBS), 17
outlier feature, 32
over-trained model, 21

1 (averaged), 45
per-channel / per-token scaling, 27
perplexity, 19
pipeline parallelism, 12
post-training quantization (PTQ), 40
precision, 4
prefill, 5
promotion, 37
pruning, 4

QServe, 43
quantization, 4
quantization noise, 30
quantization tax, 45
quantization-aware training (QAT), 38
QuaRot, SpinQuant, 32
Quartet, 38

random Hadamard transform (RHT), 38
redundant experts, 12
ReLU, 23
ReLUfication, 23
residual stream, 8
ridge point, 5
roofline model, 5
rotary position embedding (RoPE), 24
round-to-nearest, 38
router, 6
router collapse, 10
routing benefit, 13
RULER, 27

saliency, 17
salient weights, 42
scale factor, 30
selection, 25
semi-structured sparsity, 18
serving stack, 43
shared expert, 11
sigmoid gate, 8
sliding-window attention, 24
SmoothQuant, 32
SparseGPT, 19
sparsity, 4
static quantization, 43

stochastic rounding, 38
straight-through estimator (STE), 44
structured pruning, 18
structured sparsity, 6
subnormal, 29

TEAL, CATS, 23
tensor core, 18
tensor parallelism, 12
ternary weights, 44
TF32, 29
tile and block scaling, 37
token dropping, 10
tokens per parameter, 40
top-k routing, 8
total model bits, 40
total parameters, 8
Transformer Engine, 33

underflow, 35
unstructured sparsity, 7

vector quantization, 42

W8A8, W4A16, W4A8, W4A4, 43
Wanda, 19
weight-only quantization, 42
weights as shipped, 47
WGMMA, 37
width pruning, 20

z-loss, 10
ZeRO, 35
zero point, 30